



Application Hosting Configuration Guide for Cisco 8000 Series Routers, Cisco IOS XR Releases

First Published: 2020-10-15

Last Modified: 2025-09-16

Americas Headquarters

Cisco Systems, Inc.
170 West Tasman Drive
San Jose, CA 95134-1706
USA
<http://www.cisco.com>
Tel: 408 526-4000
800 553-NETS (6387)
Fax: 408 527-0883

THE SPECIFICATIONS AND INFORMATION REGARDING THE PRODUCTS IN THIS MANUAL ARE SUBJECT TO CHANGE WITHOUT NOTICE. ALL STATEMENTS, INFORMATION, AND RECOMMENDATIONS IN THIS MANUAL ARE BELIEVED TO BE ACCURATE BUT ARE PRESENTED WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED. USERS MUST TAKE FULL RESPONSIBILITY FOR THEIR APPLICATION OF ANY PRODUCTS.

THE SOFTWARE LICENSE AND LIMITED WARRANTY FOR THE ACCOMPANYING PRODUCT ARE SET FORTH IN THE INFORMATION PACKET THAT SHIPPED WITH THE PRODUCT AND ARE INCORPORATED HEREIN BY THIS REFERENCE. IF YOU ARE UNABLE TO LOCATE THE SOFTWARE LICENSE OR LIMITED WARRANTY, CONTACT YOUR CISCO REPRESENTATIVE FOR A COPY.

The Cisco implementation of TCP header compression is an adaptation of a program developed by the University of California, Berkeley (UCB) as part of UCB's public domain version of the UNIX operating system. All rights reserved. Copyright © 1981, Regents of the University of California.

NOTWITHSTANDING ANY OTHER WARRANTY HEREIN, ALL DOCUMENT FILES AND SOFTWARE OF THESE SUPPLIERS ARE PROVIDED "AS IS" WITH ALL FAULTS. CISCO AND THE ABOVE-NAMED SUPPLIERS DISCLAIM ALL WARRANTIES, EXPRESSED OR IMPLIED, INCLUDING, WITHOUT LIMITATION, THOSE OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NON-INFRINGEMENT OR ARISING FROM A COURSE OF DEALING, USAGE, OR TRADE PRACTICE.

IN NO EVENT SHALL CISCO OR ITS SUPPLIERS BE LIABLE FOR ANY INDIRECT, SPECIAL, CONSEQUENTIAL, OR INCIDENTAL DAMAGES, INCLUDING, WITHOUT LIMITATION, LOST PROFITS OR LOSS OR DAMAGE TO DATA ARISING OUT OF THE USE OR INABILITY TO USE THIS MANUAL, EVEN IF CISCO OR ITS SUPPLIERS HAVE BEEN ADVISED OF THE POSSIBILITY OF SUCH DAMAGES.

Any Internet Protocol (IP) addresses and phone numbers used in this document are not intended to be actual addresses and phone numbers. Any examples, command display output, network topology diagrams, and other figures included in the document are shown for illustrative purposes only. Any use of actual IP addresses or phone numbers in illustrative content is unintentional and coincidental.

All printed copies and duplicate soft copies of this document are considered uncontrolled. See the current online version for the latest version.

Cisco has more than 200 offices worldwide. Addresses and phone numbers are listed on the Cisco website at www.cisco.com/go/offices.

Cisco and the Cisco logo are trademarks or registered trademarks of Cisco and/or its affiliates in the U.S. and other countries. To view a list of Cisco trademarks, go to this URL: <https://www.cisco.com/c/en/us/about/legal/trademarks.html>. Third-party trademarks mentioned are the property of their respective owners. The use of the word partner does not imply a partnership relationship between Cisco and any other company. (1721R)

© 2020–2025 Cisco Systems, Inc. All rights reserved.



Preface

The *Application Hosting Configuration Guide for Cisco 8000 Series Routers* preface contains these sections:

- [Changes to This Document, on page iii](#)
- [Communications, Services, and Additional Information, on page iii](#)

Changes to This Document

This table lists the changes made to this document since it was first published.

Date	Change Summary
October 2020	Initial release of this document
May 2021	Republished for Release 7.3.15
November 2021	Republished for Release 7.5.1
April 2022	Republished for Release 7.5.2
June 2024	Republished for Release 24.2.1
December 2024	Republished for Release 24.4.1
March 2025	Republished for Release 25.1.1
September 2025	Republished for Release 25.3.1

Communications, Services, and Additional Information

- To receive timely, relevant information from Cisco, sign up at [Cisco Profile Manager](#).
- To get the business results you're looking for with the technologies that matter, visit [Cisco Services](#).
- To submit a service request, visit [Cisco Support](#).
- To discover and browse secure, validated enterprise-class apps, products, solutions and services, visit [Cisco DevNet](#).

- To obtain general networking, training, and certification titles, visit [Cisco Press](#).
- To find warranty information for a specific product or product family, access [Cisco Warranty Finder](#).

Cisco Bug Search Tool

[Cisco Bug Search Tool](#) (BST) is a web-based tool that acts as a gateway to the Cisco bug tracking system that maintains a comprehensive list of defects and vulnerabilities in Cisco products and software. BST provides you with detailed defect information about your products and software.



CHAPTER 1

New and Changed Application Hosting Features

- [New and Changed Application Hosting Features, on page 1](#)

New and Changed Application Hosting Features

Application Hosting Features Added or Modified

Feature	Description	Introduced/Changed in Release	Where Documented
CPU-Based Packet Generator	This feature was introduced.	Release 24.2.1	CPU-Based Packet Generator
Customize Docker Run Options using Application Manager	This feature was introduced.	Release 24.1.1	Customize Docker Run Options using Application Manager
Prioritize Traffic for TPAs in Sandbox Environments	This feature was introduced.	Release 24.1.1	Prioritize Traffic for TPAs in Sandbox Environments
Cisco Secure DDoS Edge Protection	This feature was introduced.	Release 24.1.1	Cisco Secure DDoS Edge Protection
Docker Application Management using IPv6 Address	This feature was introduced.	Release 7.11.1	Docker Application Management using IPv6 Address
Automatic Synchronization of Secondary IPv4 addresses from XR to Linux OS	This feature was introduced.	Release 7.5.3	Automatic Synchronization of Secondary IPv4 addresses from XR to Linux OS
Virtual IP address in the Linux networking stack	This feature was introduced.	Release 7.5.2	Verify Reachability of IOS XR and Packet I/O Infrastructure, on page 35
Support for bridge-group virtual interfaces (BVI) to be mirrored into Linux	This feature was introduced.	Release 7.2.12	Configure an Interface to be Linux-Managed, on page 45



CHAPTER 2

Application Hosting Overview

In today's networking environment, there is a need for simplifying and automating network management processes. Application hosting gives administrators a platform for leveraging their own tools and utilities for network management. Cisco IOS XR supports third-party, off-the-shelf applications that are built using Linux tool chains. With the software development kit that Cisco provides, users can cross-compile and run custom applications.

When you manage network devices with applications, you are freed of the task of focusing only on the CLI based configurations. Because of the abstraction provided by the applications, while the applications do their job, you can now focus on design and implementation aspects of the network.

The purpose of this chapter is to develop an understanding of the application hosting infrastructure, and the wide range of use cases that may be right for your need.

- [Docker Container Application Hosting, on page 3](#)
- [Hosting Third Party Applications in Sandbox Container Using Sandbox Manager, on page 16](#)
- [Top Use Cases for Application Hosting, on page 19](#)
- [Automated Deployment of Third Party Python Scripts, on page 19](#)

Docker Container Application Hosting

You can create your own container on IOS XR, and host applications within the container. The applications can be developed using any Linux distribution. Docker container application hosting is suited for applications that use system libraries that are different from those libraries provided by the IOS XR root file system.

In docker container application hosting, you can manage the amount of resources (memory and CPU) consumed by the hosted applications.

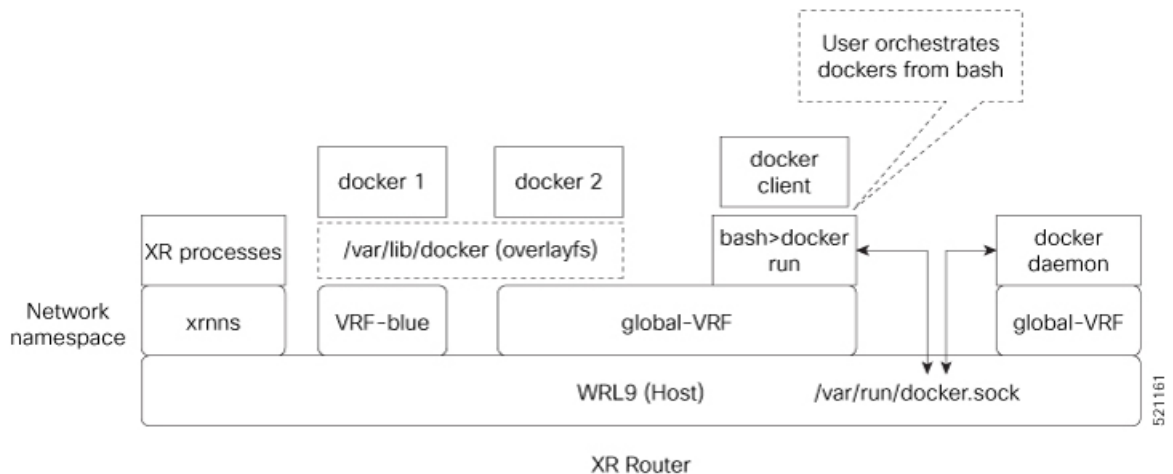
Restrictions

MPLS packets are not supported on Linux interfaces.

Docker Container Application Hosting Architecture

This section describes the docker container application hosting architecture.

Figure 1: Docker on IOS XR



The **docker client**, run from the bash shell, interacts with dockers (docker 1 and docker 2) by using the docker commands. The docker client sends the docker commands to **docker daemon**, which, then, executes the commands. The docker daemon uses the **docker.sock** Unix socket to communicate with the dockers.

When the **docker run** command is executed, a docker container is created and started from the docker image. Docker containers can be either in **global-vrf** namespace or any other defined namespace (for example, VRF-blue).

The docker utilizes overlays under the **/var/lib/docker** folder for managing the directories.

To host an application in docker containers, see [Hosting an Application in Docker Containers, on page 52](#).

App Hosting Components on IOS XR

The following are the components of App Hosting:

- **Docker on IOS XR:** The Docker daemon is included with the IOS XR software on the base Linux OS. This inclusion provides native support for running applications inside Docker containers on IOS XR. Docker is the preferred method for running TPAs on IOS XR.
- **Appmgr:** While the Docker daemon comes packaged with IOS XR, Docker applications can only be managed using appmgr. Appmgr allows users to install applications packaged as RPMs and then manage their lifecycle using the IOS XR CLI and programmable models.
- **PacketIO:** This is the router infrastructure that implements the packet path between TPAs and IOS XR running on the same router. It enables TPAs to leverage XR forwarding for sending and receiving traffic.

TPA Security

IOS XR is equipped with inherent safeguards to prevent third party applications from interfering with its role as a Network OS.

- Although IOS XR doesn't impose a limit on the number of TPAs that can run concurrently, it does impose constraints on the resources allocated to the Docker daemon, based on the following parameters:
 - **CPU:** By default, $\frac{1}{4}$ of the CPU per core available in the platform.

Starting from IOS XR Release 24.4.1, you can hard limit the default CPU usage in the range between 25-75% of the total system CPU using the **appmgr resources containers limit cpu value** command. This configuration restricts the TPAs from using more CPU than the set hard limit value irrespective of the CPU usage by other XR processes.

This example provides the CPU hard limit configuration.

```
RP/0/RSP0/CPU0:ios(config)#appmgr resources containers limit cpu ?
<25-75> In Percentage
RP/0/RSP0/CPU0:ios(config)#appmgr resources containers limit cpu 25
```

- RAM: By default, 1 GB of memory is available.

Starting from IOS XR Release 24.4.1, you can hard limit the default memory usage in the range between 1-25% of the overall system memory using the **appmgr resources containers limit memory value** command. This configuration restricts the TPAs from using more memory than the set hard limit value.

This example provides the memory hard limit configuration.

```
RP/0/RSP0/CPU0:ios(config)#appmgr resources containers limit memory ?
<1-25> In Percentage
RP/0/RSP0/CPU0:ios(config)#appmgr resources containers limit memory 20
```

- Disk space is restricted by the partition size, which varies by platform and can be checked by executing "run df -h" and examining the size of the /misc/app_host or /var/lib/docker mounts.
- All traffic to and from the application is monitored by the XR control protection, LPTS.
- Signed Applications are supported on IOS XR. Users have the option to sign their own applications by onboarding an Owner Certificate (OC) through Ownership Voucher-based workflows as described in RFC 8366. Once an Owner Certificate is onboarded, users can sign applications with GPG keys based on the Owner Certificate, which can then be authenticated during the application installation process on the router.

The table below shows the various functions performed by appmgr.

Package Manager	Lifecycle Manager	Monitoring and Debugging
• Handles installation of docker images packaged as RPMs. • Syncs the required state to standby to restart apps in cases of switchover, etc	• Handles application start/stop/kill operations. • Handles automatic application reload on: • Router reboot • Container crash • Switchover	• Logging, stats, application health check. • Forwards docker daemon logs to XR syslog. • Allows to execute into docker shell of running application.

Customize docker run options using application manager

Table 1: Feature History Table

Feature Name	Release Information	Description
Customize docker run options using application manager	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*) *This feature is supported on: • 8712-MOD-M • 8011-4G24Y4H-I
Customize docker run options using application manager	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is now supported on: • 8212-48FH-M • 8711-32FH-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
Customize docker run options using application manager	Release 24.1.1	You can now leverage Application Manager to efficiently overwrite default docker runtime configurations, tailoring them to specific parameters like CPU usage, security settings, and health checks. You can thus optimize application performance, maintain fair resource allocation among multiple dockers, and establish non-default network security settings to meet specific security requirements. Additionally, you can accurately monitor and reflect the health of individual applications. This feature modifies the docker-run-opts option command.

With this feature, runtime options for docker containerized applications on IOS-XR can be configured during launch using the **appmgr activate** command. AppMgr, which oversees docker containerized applications, ensures that these runtime options can effectively override default configurations, covering aspects like CPU, security, and health checks during the container launch.

This feature introduces multiple runtime options that allow users to customize different parameters of docker containers. The configuration of these runtime options is flexible, as users can use either command or Netconf for the configuration process. Regardless of the chosen method, runtime options must be added to **docker-run-opts** as needed.

The following are the docker run option commands introduced in IOS-XR software release 24.1.1.

Table 2: Docker Run Options

Docker Run Option	Description
--cpus	Number of CPUs
--cpuset-cpus	CPUs in which to allow execution (0-3, 0,1)
--cap-drop	Drop Linux capabilities
--user, -u	Sets the username or UID
--group-add	Add additional groups to run
--health-cmd	Run to check health
--health-interval	Time between running the check
--health-retries	Consecutive failures needed to report unhealthy
--health-start-period	Start period for the container to initialize before starting health-retries countdown
--health-timeout	Maximum time to allow one check to run
--no-healthcheck	Disable any container-specified HEALTHCHECK
--add-host	Add a custom host-to-IP mapping (host:ip)
--dns	Set custom DNS servers
--dns-opt	Set DNS options
--dns-search	Set custom DNS search domains
--domainname	Container NIS domain name
--oom-score-adj	Tune host's OOM preferences (-1000 to 1000)
--shm-size	Option to set the size of /dev/shm
--init	Run an init inside the container that forwards signals and reaps processes
--label, -l	Set meta data on a container

Docker Run Option	Description
--label-file	Read in a line delimited file of labels
--pids-limit	Tune container pids limit (set -1 for unlimited)
--work-dir	Working directory inside the container
--ulimit	Ulimit options
--read-only	Mount the container's root filesystem as read only
--volumes-from	Mount volumes from the specified container(s)
--stop-signal	Signal to stop the container
--stop-timeout	Timeout (in seconds) to stop a container
--cap-addNET_RAW	Enable NET_RAW capabilities

Prior to IOS-XR software release 24.1.1, only the below mentioned docker run option commands were supported.

Table 3: Docker Run Options

Docker Run Option	Description
--publish	Publish a container's port(s) to the host
--entrypoint	Overwrite the default ENTRYPOINT of the image
--expose	Expose a port or a range of ports
--link	Add link to another container
--env	Set environment variables
--env-file	Read in a file of environment variables
--network	Connect a container to a network
--hostname	Container host name
--interactive	Keep STDIN open even if not attached
--tty	Allocate a pseudo-TTY
--publish-all	Publish all exposed ports to random ports
--volume	Bind mount a volume
--mount	Attach a filesystem mount to the container
--restart	Restart policy to apply when a container exits
--cap-add	Add Linux capabilities

Docker Run Option	Description
--log-driver	Logging driver for the container
--log-opt	Log driver options
--detach	Run container in background and print container ID
--memory	Memory limit
--memory-reservation	Memory soft limit
--cpu-shares	CPU shares (relative weight)
--sysctl	Sysctl options

Guidelines and Limitations

- For the options `--mount` and `--volume`, only the following values can be configured:
 - `"/var/run/netns"`
 - `"/var/lib/docker"`
 - `"/misc/disk1"`
 - `"/disk0"`
- The maximum allowed size for `shm-size` option is 64 Mb.
- Prior to Release 24.4.1, all container logs were recorded with an info severity level (sev-6), regardless of the docker run time options used. From Release 24.4.1, if the docker run time option `-it` is used, the container logs are generated with an info severity level (sev-6). However, if the `--it` option is not included, the logs are produced with an error severity level (sev-3).
- From Release 24.4.1, you can use the rsyslog daemon to forward syslog messages to remote syslog servers. To know more, see [Support for logging functionality on third-party applications](#).

Configuration

This section provides the information on how to configure the docker run time options.

In this example we configure the docker run time option `--pids-limit` to limit the number of process IDs using `appmgr`.

```
Router#appmgr application alpine_app activate type docker source alpine docker-run-opts
"-it -pids-limit 90" docker-run-cmd "sh"
Router#
```

In this example we configure the docker run time option `--pids-limit` to limit the number of process IDs using `Netconf`.

```
<rpc xmlns="urn:ietf:params:xml:ns:netconf:base:1.0" message-id="101">
  <edit-config>
    <target>
      <candidate/>
    </target>
  </edit-config>
</rpc>
```

```

<appmgr xmlns=http://cisco.com/ns/yang/Cisco-IOS-XR-um-appmgr-cfg>

  <applications>
    <application>
      <application-name>alpine_app</application-name>
      <activate>
        <type>docker</type>

                                <source-name>alpine</source-name>
                                <docker-run-cmd>/bin/sh</docker-run-cmd>
                                <docker-run-opts>-it

--pids-limit=90</docker-run-opts>
      </activate>
    </application>
  </applications>
</appmgr>
</config>
</edit-config>

```

Verification

This example shows how to verify the docker run time option configuration.

```

Router# show running-config appmgr
Thu Mar 23 08:22:47.014 UTC
appmgr
  application alpine_app
    activate type docker source alpine docker-run-opts "-it -pids-limit 90" docker-run-cmd
    "sh"
  !
!

```

You can also use **docker inspect** *container id* to verify the docker run time option configuration.

```

Router# docker inspect 25f3c30eb424
[
  {
    "PidsLimit": 90,
  }
]

```

Prioritize Traffic for TPAs in Sandbox Environments

Table 4: Feature History Table

Feature Name	Release Information	Description
Prioritize Traffic for TPAs in Sandbox Environments	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*) *This feature is supported on: • 8712-MOD-M • 8011-4G24Y4H-I

Feature Name	Release Information	Description
Prioritize Traffic for TPAs in Sandbox Environments	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is now supported on: • 8212-48FH-M • 8711-32FH-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
Prioritize Traffic for TPAs in Sandbox Environments	Release 24.1.1	You can now optimize network performance, implement traffic segregation, and prevent packet drops due to congestion for Third Party Application (TPA) within the Sandbox environment, improving reliability and efficiency. This is achieved through enhanced LPTS-based traffic prioritization for TPAs hosted within a sandbox container. This feature introduces these changes: CLI: • sandbox flow TPA-APPMGR-HIGH ports • sandbox flow TPA-APPMGR-MEDIUM ports • sandbox flow TPA-APPMGR-LOW ports

With this enhancement, you have the flexibility to categorize traffic flows from TPAs hosted in a sandbox based on priority levels, offering better granular control over traffic handling. Prior to this release, traffic from TPAs hosted in a sandbox flowed through a single queue, leading to policer overload and subsequent packet drop.

Configuring Traffic Prioritization for TPA in a Sandbox

During the configuration of a TPA port, you can now set the priority for the port as High, Medium, or Low.

Configuring high priority traffic port

This example shows how to configure TPA traffic in port 2018 to high LPTS flow priority.

```
Router(config)# sandbox flow TPA-APPMGR-HIGH ports 2018
```

Configuring medium priority traffic port

This example shows how to configure TPA traffic in port 6666 to medium LPTS flow priority.

```
Router(config)# sandbox flow TPA-APPMGR-MEDIUM ports 6666
```

Configuring low priority traffic port

This example shows how to configure TPA traffic in port 60100 to low LPTS flow priority.

```
Router(config)# sandbox flow TPA-APPMGR-LOW ports 60100
```

Verification

This example shows how to verify TPA traffic prioritization.

```
Router(config)# show lpts pifib hardware police location
```

TPA-APPMGR-HIGH 0	103	np	NPU	1940	1000	0	0
TPA-APPMGR-HIGH 1	103	np	NPU	1940	1000	1456	0
TPA-APPMGR-MED 0	104	np	NPU	1940	1000	0	0
TPA-APPMGR-MED 1	104	np	NPU	1940	1000	1455	0
TPA-APPMGR-LOW 0	105	np	NPU	1940	1000	0	0
TPA-APPMGR-LOW 1	105	np	NPU	1940	1000	1456	0

Docker Application Management using IPv6 Address

Table 5: Feature History Table

Feature Name	Release Information	Description
Docker Application Management using IPv6 Address	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*) *This feature is supported on: • 8712-MOD-M • 8011-4G24Y4H-I

Feature Name	Release Information	Description
Docker Application Management using IPv6 Address	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is now supported on: • 8212-48FH-M • 8711-32FH-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
Docker Application Management using IPv6 Address	Release 7.11.1	In this release, you gain the ability to manage Docker applications within containers using IPv6 addresses via the router's management interface. Leveraging IPv6 addresses provides expanded addressing options, enhances network scalability, and enables better segmentation and isolation of applications within the network. Prior to this update, only IPv4 addresses could be used to manage docker applications.

The Application Manager in IOS-XR software release 7.3.15 introduces support for an application networking feature that facilitates traffic forwarding across Virtual Routing and Forwarding (VRF) instances. This feature is implemented through the deployment of a relay agent contained within an independent docker container.

The relay agent acts as a bridge, connecting two network namespaces within the host system and actively transferring traffic between them. Configurations can be made to establish forwarding between either a single pair of ports or multiple pairs, based on your network requirements.

One of the main uses of this feature is to allow the management of Linux-based Docker applications that are running in the default VRF through a management interface. This management interface can be located in a separate VRF. This feature ensures that Docker applications can be managed seamlessly across different VRFs.

In the IOS-XR software release 7.11.1, enhanced management capabilities are offered for docker applications. Now, you can leverage IPv6 addresses to manage applications within docker containers via the management interface of the Cisco 8000 router. This update provides improved accessibility and control over your Docker applications using IPv6 addressing. Prior to the IOS-XR software release 7.11.1, application management for docker containers could only be conducted using IPv4 addresses.

Restrictions and Limitations

In configuring your setup, please consider the following restrictions and limitations:

- **VRF Forwarding Limitation:** The Virtual Routing and Forwarding (VRF) is only supported for Docker apps with host networking.
- **Relay Agent Availability and Management:** The relay agent container is designed to be highly available. It will be managed by the Application Manager (App Mgr).
- **Relay Agent Creation:** For each pair of forwarded ports, one relay agent container will be created.
- **Port Limitation per Application:** The total effective number of ports for each application is limited to a maximum of 10.

Configure VRF Forwarding

To manage a Docker application using the Application Manager through the Management Interface, follow these steps:

Procedure

- Step 1** **Configure the app manager:** The application manager is configured to access the docker application. Use the **appmgr application** keyword to enable and specify configuration parameters for the VRF forwarding. A typical example would look like this:

Example:

```
Router#appmgr
Router#application Testapp
```

Note

The VRF forwarding related run options like **--vrf-forward** and **--vrf-forward-ip-range** will not be passed to the Docker engine when the app container is run.

- Step 2** **Enable Basic Forwarding Between Two Ports:** To enable traffic forwarding between two ports in different VRFs, use the following configuration:

Example:

```
Router#activate type docker source swanagent docker-run-opts "--vrf-forward vrf-mgmt:5001
vrf-default:8001 --net=host -it"
```

This command enables traffic on port 5000 at all addresses in vrf-mgmt to be forwarded to the destination veth device in vrf-default on port 8000.

To enable VRF forwarding between multiple ports, follow the steps below:

- **Enable Forwarding Between a Range of Ports:** To enable traffic forwarding between port ranges in different VRFs, use the following configuration:

```
Router#--vrf-forward vrf-mgmt:5000-5002 vrf-default:8000-8002
```

This command enables traffic on ports 5000, 5001, and 5002 at all addresses in vrf-mgmt to be forwarded to the destination veth device in vrf-default on ports 8000, 8001, and 8002 respectively.

- **Enable Forwarding Between Multiple VRF Pairs or Port Ranges:** To enable traffic forwarding between multiple VRF pairs, use multiple `--vrf-forward` command.

```
Router#--vrf-forward vrf-mgmt:5000 vrf-default:8000 --vrf-forward vrf-mgmt:5003-5004
vrf-default:8003-8004
```

```
Router#--vrf-forward vrf-mgmt1:5000 vrf-default:8000 --vrf-forward vrf-mgmt2:5000 vrf-default:8001
```

You can provide any number of `--vrf-forward` options, but the total number of port pairs involved should not exceed 10.

Verifying VRF Forwarding for Application Manager

Use the **show appmgr application name** keyword to verify the VRF forwarding. A typical example would look like this:

```
RP/0/RP0/CPU0:ios#show appmgr application name swan info detail
Thu Oct 26 11:59:32.798 UTC
Application: swan
  Type: Docker
  Source: swanagent
  Config State: Activated
  Docker Information:
    Container ID: f230a2396b85f6b3eeb01a8a4450a47e5bd8499fe5cfd141c2d0fba905b63ec
    Container name: swan
    Labels:
original-path: /var/lib/docker/containers/f230a2396b85f6b3eeb01a8a4450a47e5bd8499fe5cfd141c2d0fba905b63ec/swan
    Image: swancr.azurecr.io/swanagentxr-iosxr:2.4.0-0ebd435
    Command: "./agentxr"
    Created at: 2023-10-26 11:58:45 +0000 UTC
    Running for: 48 seconds ago
    Status: Up 47 seconds
    Size: 0B (virtual 29.3MB)
    Ports:
    Mounts:
    /var/lib/docker/appmgr/config/swanagent/hostname, /var/lib/docker/appmgr/config/swanagent, /var/lib/docker/ems/grpc.sock, /var/run/netns

Networks: host
LocalVolumes: 0
Vrf Relays:
  Vrf Relay: vrf_relay.swan.6a98f0ed060bffa
    Source VRF: vrf-management
    Source Port: 11111
    Destination VRF: vrf-default
    Destination Port: 10000
    IP Address Range: 172.16.0.0/12
    Status: Up 45 seconds
```

Use the **show running-config appmgr** keyword to check the running configuration.

```
Router#show running-config appmgr
Thu Oct 26 12:04:06.063 UTC
appmgr
  application swan
    activate type docker source swanagent docker-run-opts "--vrf-forward vrf-management:11111
vrf-default:10000 -it --restart always --cap-add=SYS_ADMIN --net=host --log-opt max-size=20m
--log-opt max-file=3 -e HOSTNAME=$HOSTNAME -v /var/run/netns:/var/run/netns -v
{app_install_root}/config/swanagent:/root/config -v
{app_install_root}/config/swanagent/hostname:/etc/hostname -v
/var/lib/docker/ems/grpc.sock:/root/grpc.sock"
```

Hosting Third Party Applications in Sandbox Container Using Sandbox Manager

Table 6: Feature History Table

Feature Name	Release Information	Feature Description
Hosting Third Party Applications in Sandbox Container Using Sandbox Manager	7.5.3	This release introduces Sandbox Manager for hosting and functioning third-party client application in the CentOS 8 based Sandbox Container. The Sandbox container supports configuration, deployment, and management of third-party client applications from the third-party server. The Sandbox manger uses IOS XR commands for managing the Sandbox container.

The Sandbox container enables you to configure, deploy, and manage third-party client applications through the respective third-party server over a network. The Sandbox manager activates the Sandbox container using the APPMGR client library APIs. During the router bootup, the third-party client applications are placed in the Sandbox container using ZTP and get activated when the sandbox manger is enabled. The third-party client applications can then connect to the respective server for installing or upgrading applications in the Sandbox container. Sandbox container operates on CentOS 8, this enables you to control the applications in the container using the docker commands. All the activated third-party client applications can restart automatically after a router reload or an RP switchover.

Supported Commands on Sandbox Manager

This section describes the operations and the IOS XR commands that are supported on the sandbox manager:

- **Enable and disable sandbox manager:** This command is used to enable or disable sandbox manager:

- **Enable—**

The following command enables the Sandbox Manager:

```
RP/0/RP0/CPU0:ios#conf
RP/0/RP0/CPU0:ios(config)#sandbox enable
RP/0/RP0/CPU0:ios(config)#commit
```

- **Disable—**

The following command disables the Sandbox Manager:

```
RP/0/RP0/CPU0:ios#conf
RP/0/RP0/CPU0:ios(config)# no sandbox enable
RP/0/RP0/CPU0:ios(config)#commit
```

- **TPA traffic flow prioritization:** These commands are used to configure traffic priority for third party applications within a Sandbox container:

- **High priority traffic—**

The following command configures TPA traffic in port 2018 to high LPTS flow priority

```
Router(config)# sandbox flow TPA-APPMGR-HIGH ports 2018
```

- Medium priority traffic—

The following command configures TPA traffic in port 6666 to medium LPTS flow priority

```
Router(config)# sandbox flow TPA-APPMGR-MEDIUM ports 6666
```

- Low priority traffic—

The following command configures TPA traffic in port 60100 to low LPTS flow priority

```
Router(config)# sandbox flow TPA-APPMGR-LOW ports 60100
```

- **Show commands**

- Info—

The following command shows the Sandbox Manager and application info:

```
RP/0/RP0/CPU0:ios#show sandbox info
Thu Jun 30 06:56:45.593 UTC
```

```
Sandbox Config State: Enabled
```

```
APP INFO:
```

```
Image: /pkg/opt/cisco/XR/appmgr/images/sandbox-centos.tar.gz
Config state: Activated
Container state: Running
```

- Detail—

The following command shows the Sandbox Manager and application details:

```
RP/0/RP0/CPU0:ios#show sandbox detail
Thu Jun 30 06:57:46.724 UTC
```

```
Sandbox Config State: Enabled
```

```
APP INFO:
```

```
Image: /pkg/opt/cisco/XR/appmgr/images/sandbox-centos.tar.gz
```

```
Run Options:
```

```
--restart always
--cap-add SYS_ADMIN --cap-add NET_ADMIN
--log-opt max-size=10m --log-opt max-file=3
--net host
--mount type=bind,source=/sys/fs/cgroup,target=/sys/fs/cgroup,readonly
--mount type=bind,source=/var/run/netns,target=/netns,bind-propagation=shared
--mount type=bind,source=/opt/sandbox,target=/opt/sandbox,bind-propagation=shared

--mount type=bind,source=/misc/disk1/sandbox,target=/host,bind-propagation=shared
```

```
Config state: Activated
Container state: Running
```

```
STATS INFO:
```

```
Cpu Percentage: 0.01%
Memory Usage: 13.57MiB / 19.42GiB
Net IO: 0B / 0B
Block IO: 0B / 1.2MB
Memory Percentage: 0.07%
pids: 2
```

• Services—

The following command shows the Sandbox Manager and application services:

```
RP/0/RP0/CPU0:ios#show sandbox services
Wed Jul  6 05:59:16.446 UTC
UNIT                                LOAD    ACTIVE SUB    DESCRIPTION
-.mount                             loaded active mounted /
dev-mqueue.mount                    loaded active mounted POSIX Message Queue File Sys
etc-hostname.mount                  loaded active mounted /etc/hostname
etc-hosts.mount                     loaded active mounted /etc/hosts
etc-resolv.conf.mount                loaded active mounted /etc/resolv.conf
host.mount                           loaded active mounted /host
netns-default.mount                 loaded active mounted /netns/default
netns-global\x2dvrf.mount            loaded active mounted /netns/global-vrf
netns-vrf\x2dblue.mount              loaded active mounted /netns/vrf-blue
netns-vrf\x2ddefault.mount            loaded active mounted /netns/vrf-default
netns-vrf\x2dmanagement.mount         loaded active mounted /netns/vrf-management
netns-vrf\x2dred.mount                loaded active mounted /netns/vrf-red
netns-xrnns.mount                   loaded active mounted /netns/xrnns
netns.mount                          loaded active mounted /netns
proc-acpi.mount                     loaded active mounted /proc/acpi
proc-bus.mount                       loaded active mounted /proc/bus
proc-fs.mount                       loaded active mounted /proc/fs
proc-irq.mount                      loaded active mounted /proc/irq
proc-kcore.mount                    loaded active mounted /proc/kcore
proc-keys.mount                     loaded active mounted /proc/keys
proc-latency_stats.mount             loaded active mounted /proc/latency_stats
proc-sched_debug.mount               loaded active mounted /proc/sched_debug
proc-scsi.mount                     loaded active mounted /proc/scsi
proc-sysrq\x2dtrigger.mount           loaded active mounted /proc/sysrq-trigger
proc-timer_list.mount                loaded active mounted /proc/timer_list
sys-firmware.mount                  loaded active mounted /sys/firmware
systemd-journald.service             loaded active running Journal Service
systemd-tmpfiles-setup.service        loaded active exited Create Volatile Files and
Di
-.slice                             loaded active active Root Slice
system.slice                         loaded active active System Slice
dbus.socket                          loaded active listening D-Bus System Message Bus Soc
systemd-journald.socket               loaded active running Journal Socket
systemd-shutdown.socket               loaded active listening Delayed Shutdown Socket
basic.target                          loaded active active Basic System
local-fs.target                       loaded active active Local File Systems
multi-user.target                     loaded active active Multi-User System
paths.target                         loaded active active Paths
slices.target                         loaded active active Slices
sockets.target                       loaded active active Sockets
swap.target                          loaded active active Swap
sysinit.target                       loaded active active System Initialization
timers.target                         loaded active active Timers
systemd-tmpfiles-clean.timer           loaded active waiting Daily Cleanup of Temporary
D

LOAD    = Reflects whether the unit definition was properly loaded.
ACTIVE  = The high-level unit activation state, i.e. generalization of SUB.
SUB     = The low-level unit activation state, values depend on unit type.

43 loaded units listed. Pass --all to see loaded but inactive units, to
o.
To show all installed unit files use 'systemctl list-unit-files'.
```

• Access Sandbox—

The following command is used to access sandbox container:

```
RP/0/RP0/CPU0:ios#bash sandbox
root@ios:/data# exit
exit
RP/0/RP0/CPU0:ios#
```

- Linux commands—

The following command is used to run linux commands inside sandbox container:

```
RP/0/RP0/CPU0:ios#bash sandbox -c linux-command
RP/0/RP0/CPU0:ios#
```

Top Use Cases for Application Hosting

Some of the top use cases for application hosting are:

- **Measure Network Performance:** An application can be hosted to measure the bandwidth, throughput and latency of the network and monitor the performance. An example of such an application is the iPerf tool.
- **Automate Server Management:** An application can be hosted to automate the server functions like upgrading software, allocation of resources, creating user accounts, and so on. Examples of such an application are the Chef and Puppet configuration management tools.

Automated Deployment of Third Party Python Scripts

Feature Name	Release Information	Description
Automated Deployment of Third Party Python Scripts	Release 24.2.1	When you deploy custom or third-party Python scripts on routers running IOS XR software using third-party RPMs, these scripts are automatically executed. This streamlines the deployment process and enhances the speed of script execution. Traditionally, script deployment required an external controller, which used interfaces like NETCONF, SNMP, and SSH to communicate with the router. This feature eliminates the need for such external controllers, simplifying the workflow and improving efficiency.

Efficient network automation is pivotal in handling extensive cloud-computing networks. The Cisco IOS XR infrastructure plays a crucial role by enabling automation through the initiation of API calls and execution of scripts. Traditionally, an external controller is used for this purpose, utilizing interfaces like NETCONF, SNMP, and SSH to communicate with the router.

This feature streamlines the operational structure by executing automation scripts directly on the router, thus eliminating the need for an external controller. It allows scripts to leverage Python libraries and access underlying router information. This approach not only accelerates the execution of various types of scripts but also enhances reliability by removing dependencies on the speed and network reachability of an external controller.

The third party script is automatically executed by the `xr_script_scheduler.py` script upon the installation of third-party RPMs. No specific configuration is required to run these scripts after installation.

The below steps provide the information on how to deploy and activate third party script:

Procedure

- Step 1** Adding and Activating Scheduler Script - Add and activate scheduler script in in-built script repository - Copy the "xr_script_scheduler.py" scheduler script to the In-Built Script Repository, and simultaneously activate it using the following commands:

Example:

```
cp /path/to/xr_script_scheduler.py /opt/cisco/install-iosxr/base/opt/cisco/ops-script-repo/process/
appmgr activate script name xr_script_scheduler.py
Router#
```

Replace "/path/to/xr_script_scheduler.py" with the actual path of the script. This command copies the script to the specified directory and activates it in the XR configuration mode.

This step ensures the script is added to the repository and activated for continuous execution.

- Step 2** Verify the Status of Scheduler Script - To confirm the availability of the scheduler script, run the following command on the router.

Example:

```
Router# show script status
Tue Oct 24 18:03:09.220 UTC
```

Name	Type	Status	Last Action	Action Time
show_interfaces_counters_ecn.py	exec	Ready	NEW	Tue Oct 24 07:10:36 2023
xr_data_collector.py	exec	Ready	NEW	Tue Oct 24 07:10:36 2023
xr_script_scheduler.py	process	Ready	NEW	Tue Oct 24 07:10:36 2023

```
Router#
```

Ensure that the output displays "Ready" for the "xr_script_scheduler.py" script, indicating that the script checksum is verified and it is ready to run. This single step provides a quick verification of the scheduler script's status.

- Step 3** Configure appmgr to Automatically Run the Scheduler Script - Activate the scheduler script automatically using the "autorun" option with the following configuration:

Example:

```
Router(config)#appmgr
```



```
Router(config-appmgr)#process-script xr_script_scheduler
Router(config-process-script)#executable xr_script_scheduler.py
Router(config-process-script)#autorun
Router(config-process-script)#commit
```

The 'autorun' configuration has been added to enable automatic activation of the process script. If you prefer manual activation/deactivation using cli, the 'autorun' configuration line can be skipped.

Step 4

Verify scheduler script is running - To verify if the scheduler script is running, execute the **show script execution** command. This command will display a list of OPS scripts currently running. If the scheduler script has been correctly configured and activated, the scheduler script execution detail will appear in the output.

Example:

```
Router# show script execution
Tue Oct 24 18:01:56.590 UTC
```

```
=====
Req. ID   | Name (type)                               | Start                               | Duration   |
Return | Status
-----
1698170509| xr_script_scheduler.py (process)          | Tue Oct 24 18:01:49 2023 | 7.68s      | None
      | Started
-----

Execution Details:
-----
Script Name   : xr_script_scheduler.py
Version      : 7.3.6.14Iv1.0.0
Log location  : /harddisk:/mirror/script-mgmt/logs/xr_script_scheduler.py_process_xr_script_scheduler

Arguments    :
Run Options  : Logging level - INFO, Max. Runtime - 0s, Mode - Background
Events:
-----
1.  Event      : New
    Time       : Tue Oct 24 18:01:49 2023
    Time Elapsed : 0.00s Seconds
    Description  : Started by Appmgr
2.  Event      : Started
    Time       : Tue Oct 24 18:01:49 2023
    Time Elapsed : 0.11s Seconds
    Description  : Script execution started. PID (15985)
=====
```

```
Router#
```

Step 5

Transfer of Third-Party RPM with Debug/Monitoring Scripts - Transfer the third-party RPM containing debug/monitoring scripts onto the router. This RPM includes Python scripts for debugging/monitoring and a run parameters JSON file.

Example:

```
Router# scp user@171.68.251.248:/users/savinaya/rpm-factory/RPMS/x86_64/nms-1.1-24.1.1.x86_64.rpm
/harddisk:

Tue Oct 24 18:02:42.400 UTC
<snip>
Password:
nms-1.1-24.1.1.x86_64.rpm                               100% 9664    881.5KB/s    00:00

Router#
Router# dir harddisk:/nms-1.1-24.1.1.x86_64.rpm
```

Step 6

Install the third-party RPM - Use the **appmgr package install** CLI command for the installation of the RPM.

Example:

```
Router# appmgr package install rpm /harddisk:/nms-1.1-24.1.1.x86_64.rpm
Tue Oct 24 18:03:26.685 UTC
Router# show appmgr packages installed
Tue Oct 24 19:42:07.967 UTC
Sno Package
-----
1 nms-1.1-24.1.1.x86_64
Router#
```

Step 7 Verify the operation of the debug/monitoring scripts - You can verify that these scripts are functioning by executing the **show script execution** command.

Example:

```
Router# show script execution
Tue Oct 24 19:41:15.882 UTC
```

Req. ID	Name (type)	Start	Duration	
Return	Status			
1698176223	xr_script_scheduler.py (process)	Tue Oct 24 19:37:02 2023	253.32s	None
	Started			
1698176224	nms/monitor_int_rx_cntr.py (exec)	Tue Oct 24 19:38:43 2023	152.46s	None
	Started			
1698176225	nms/monitor_int_rx_cntr.py (exec)	Tue Oct 24 19:38:44 2023	152.03s	None
	Started			
1698176226	nms/monitor_int_rx_cntr2.py (exec)	Tue Oct 24 19:38:44 2023	151.63s	None
	Started			

```
Router#
```

Step 8 Stopping the scheduler script - Stop the scheduler using the **appmgr process-script stop** command.

Example:

```
Router# show script execution
Tue Oct 24 20:04:22.021 UTC
```

Req. ID	Name (type)	Start	Duration	
Return	Status			
1698176224	nms/monitor_int_rx_cntr.py (exec)	Tue Oct 24 19:38:43 2023	234.21s	-9
	Stopped			
1698176225	nms/monitor_int_rx_cntr.py (exec)	Tue Oct 24 19:38:44 2023	234.43s	-9
	Stopped			
1698176226	nms/monitor_int_rx_cntr2.py (exec)	Tue Oct 24 19:38:44 2023	234.67s	-9
	Stopped			
1698176227	ops/monitor_int_rx_cntr3.py (exec)	Tue Oct 24 19:41:35 2023	97.56s	-9
	Stopped			
1698176228	ops/monitor_int_rx_cntr4.py (exec)	Tue Oct 24 19:41:36 2023	97.19s	-9
	Stopped			
1698176229	ops/monitor_int_rx_cntr5.py (exec)	Tue Oct 24 19:41:36 2023	96.48s	-9
	Stopped			
1698176231	ops/monitor_int_rx_cntr3.py (exec)	Tue Oct 24 19:43:44 2023	760.88s	-9
	Stopped			
1698176232	ops/monitor_int_rx_cntr4.py (exec)	Tue Oct 24 19:43:44 2023	760.53s	-9
	Stopped			
1698176233	ops/monitor_int_rx_cntr5.py (exec)	Tue Oct 24 19:43:44 2023	760.20s	-9
	Stopped			

```

1698176234| nms/monitor_int_rx_cntr.py (exec) | Tue Oct 24 19:44:15 2023 | 202.88s | -9
| Stopped
1698176235| nms/monitor_int_rx_cntr.py (exec) | Tue Oct 24 19:44:15 2023 | 203.01s | -9
| Stopped
1698176236| nms/monitor_int_rx_cntr2.py (exec) | Tue Oct 24 19:44:16 2023 | 203.17s | -9
| Stopped
1698176237| nms/monitor_int_rx_cntr.py (exec) | Tue Oct 24 19:53:41 2023 | 163.99s | -9
| Stopped
1698176238| nms/monitor_int_rx_cntr.py (exec) | Tue Oct 24 19:53:41 2023 | 163.52s | -9
| Stopped
1698176239| nms/monitor_int_rx_cntr2.py (exec) | Tue Oct 24 19:53:42 2023 | 163.11s | -9
| Stopped
1698176252| xr_script_scheduler.py (process) | Tue Oct 24 20:00:20 2023 | 220.61s | -15
| Stopped
1698176253| nms/monitor_int_rx_cntr.py (exec) | Tue Oct 24 20:00:21 2023 | 222.11s | -9
| Stopped
1698176254| nms/monitor_int_rx_cntr.py (exec) | Tue Oct 24 20:00:21 2023 | 221.76s | -9
| Stopped
1698176255| nms/monitor_int_rx_cntr2.py (exec) | Tue Oct 24 20:00:22 2023 | 221.39s | -9
| Stopped
1698176256| ops/monitor_int_rx_cntr3.py (exec) | Tue Oct 24 20:00:22 2023 | 221.08s | -9
| Stopped
1698176257| ops/monitor_int_rx_cntr4.py (exec) | Tue Oct 24 20:00:23 2023 | 131.46s | -9
| Stopped
1698176258| ops/monitor_int_rx_cntr5.py (exec) | Tue Oct 24 20:00:23 2023 | 220.30s | -9
| Stopped

```

Router#



CHAPTER 3

Cisco Secure DDoS Edge Protection

Distributed denial of service (DDoS) edge protection stops DDoS attacks at the network entry point.

Table 7: Feature History Table

Feature Name	Release Information	Description
Cisco Secure DDoS Edge Protection	Release 25.2.1	Introduced in this release on: Fixed Systems (8200 [ASIC: Q200, P100], 8700 [ASIC: P100, K100], 8010 [ASIC: A100]) (select variants only*); Centralized Systems (8600 [ASIC: Q200]); Modular Systems (8800 [LC ASIC: Q100, Q200, P100]) (select variants only*) You can now enable the router to detect DDoS attacks targeting MPLS traffic using DDoS edge protection. The router analyzes MPLS flows to identify malicious traffic patterns, ensuring the availability and performance of services traversing MPLS networks.
Cisco Secure DDoS Edge Protection	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*) *This feature is supported on: • 8712-MOD-M • 8011-4G24Y4H-I

Feature Name	Release Information	Description
Cisco Secure DDoS Edge Protection	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is supported on: • 8212-48FH-M • 8711-32FH-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
Cisco Secure DDoS Edge Protection	Release 24.1.1	You can now efficiently block malicious traffic, safeguarding your network's performance and availability. This is achieved as we have implemented protection against distributed denial-of-service (DDoS) attacks at the network edge, strategically deployed at the ingress point where external network traffic enters. A centralized controller manages DDoS mitigation capabilities using information from a collection of detectors deployed on the routers. These detectors analyze IPv4 and IPv6 traffic in real-time to identify DDoS attacks. Upon detection, the controller enforces deny ACLs to block malicious traffic while allowing legitimate traffic. This local inspection enhances visibility, speeds up response times, and optimizes the network without the need for additional hardware or attack traffic redirection.

The Cisco secure DDoS edge protection software actively halts DDoS attacks at the network entry point, enabling immediate response to threats. Positioned at the network edge, it identifies and counteracts DDoS threats directly on the router. This strategy minimizes network and application impact without affecting core bandwidth by avoiding backhaul of malicious traffic.

Components of Cisco secure DDoS edge protection

The Cisco secure DDoS edge protection consists of these components:

- A highly available centralized controller that manages a collection of detectors deployed on routers. The controller can be cloud-based or on-premises. Key functions of the controller include
 - managing the container lifecycle for detectors
 - configuring and editing detector profiles and security settings
 - checking detector health, displaying real-time attack forensics and threat intelligence analyses
 - controlling DDoS attack mitigation at the network ingress point
 - providing real-time and historical event reporting, and
 - operational control and incident response.
- A collection of detectors that are deployed on edge or peering routers. The detector is a resource-efficient application container for real-time DDoS detection deployed on routers and managed by the centralized controller.

It analyzes IPv4 and IPv6 traffic on each ingress interface to identify DDoS attacks as they occur.

Starting from Cisco IOS XR Release 25.2.1, MPLS traffic is analyzed using the extracted information from the IPv4 and IPv6 payload to detect accurate DDoS attacks.

Upon detecting a DDoS attack, the centralized controller promptly begins mitigation. The mitigation includes enforcing a deny ACL to block the attack traffic while still allowing legitimate traffic to pass through.

Benefits of Cisco secure DDoS edge protection

Cisco secure DDoS edge protection have the following benefits:

- Stops DDoS attacks at the network ingress
- Requires no additional hardware or facilities such as power, rack space, and cooling
- Requires no changes to the architecture
- Avoids the need to overprovision network facilities such as links and routers to account for attack traffic
- Prevents backhauling of malicious traffic
- Minimizes network outages and optimizes the end-user experience, and
- Meets low-latency application requirements.
- [Prerequisites for installing DDoS edge protection, on page 27](#)
- [Restrictions of DDoS edge protection, on page 28](#)
- [Install and configure DDoS edge protection, on page 28](#)
- [Verify DDoS edge protection application configuration, on page 30](#)

Prerequisites for installing DDoS edge protection

- Configure the management interface to reach the DDoS controller IP address.
- Manually configure the base ACL, NetFlow, and SSH configurations.

Restrictions of DDoS edge protection

- The DDoS edge protection does not support tunnel traffic.
- The system supports only the default VRF configuration, and it applies solely to the management port. For effective communication between the Docker and the controller, you must configure the management port to operate within the default VRF exclusively. This setup guarantees that the Docker can reliably interact with the controller without any network interruptions.
- The version protobuf of **flow exporter-map** is only supported for **Monitor Map** configured with **record mpls ipv4-ipv6-fields**.

Install and configure DDoS edge protection

You can install the DDoS edge protection application through the DDoS edge protection controller. Perform the following:

1. Install and download the DDoS edge protection controller software package from the [Software Download](#) page. You can access the user interface, when the controller installation is complete.

Log in to the controller services instance to monitor, manage, and control the device.

2. Configure a Loopback on the router.

```
Router(config)#interface Loopback100
Router(config-if)# ipv4 address 15.1.1.2 255.255.255.255
Router(config-if)# exit
Router(config)#interface Loopback101
Router(config-if)# ipv4 address 17.1.1.2 255.255.255.255
Router(config-if)#commit
```

3. Configure an ACL on the router.

```
Router(config)#ipv4 access-list myACL
Router(config-ipv4-acl)# 1301 permit ipv4 any any
Router(config-ipv4-acl)# exit
Router(config)#ipv6 access-list myACL
Router(config-ipv6-acl)# 1301 permit ipv6 any any
Router(config-ipv6-acl)#exit
Router(config)#commit
```

For more information on implementing access lists and prefix lists, see [Understanding Access-List](#).

If there is any DDoS attack, the controller performs the mitigation action using the ACL rule automatically.

The following is a sample configuration to deny DDoS attacker traffic using user defined ACE rule:

```
1 deny udp any eq 19 host 45.0.0.1 eq 0 packet-length eq 128 ttl eq 64
2 deny tcp any host 45.0.0.1 eq www match-all -established -fin -psh +syn -urg
packet-length eq 60 ttl eq 64
1301 permit ipv4 any any
```

Result: Configuration updates are sent by the controller to the router.

4. Configure SSH on the router.

```
Router(config)#ssh server v2
Router(config)#ssh server netconf
```



```

Router(config)#netconf agent tty
Router(config-netconf-tty)#netconf-yang agent ssh
Router(config)#ssh timeout 120
Router(config)#ssh server rate-limit 600
Router(config)#ssh server session-limit 110
Router(config)#ssh server v2
Router(config)#ssh server vrf default
Router(config)#ssh server netconf vrf default
Router(config)#commit

```

5. Execute the **ping** command on the router and check the router connection to the DDoS controller.

```

Router#ping 10.105.237.54
Thu Jun 1 07:16:43.654 UTC
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 10.105.237.54 timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 2/2/4 ms
RP/0/RP0/CPU0:Router#bash
Thu Jun 1 07:16:53.024 UTC
[Router:~]$ping 10.105.237.54
PING 10.105.237.54 (10.105.237.54) 56(84) bytes of data.
64 bytes from 10.105.237.54: icmp_seq=1 ttl=63 time=1.73 ms
64 bytes from 10.105.237.54: icmp_seq=2 ttl=63 time=1.29 ms
64 bytes from 10.105.237.54: icmp_seq=3 ttl=63 time=1.27 ms
64 bytes from 10.105.237.54: icmp_seq=4 ttl=63 time=1.75 ms
^C
--- 10.105.237.54 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 1.270/1.510/1.751/0.230 ms
[Router:~]$

```

6. Enter the details of the device into the DDoS edge protection controller panel and verify that the Deployment, Container, and Configuration indicators all display green.

For more information on installing the DDoS controller, see the [Cisco Secure DDoS Edge Protection Installation guide](#).

The controller automatically performs the following netflow configuration on the router:

```

//Configuring Monitor Map
Router(config)#flow monitor-map DetectPro_Monitor_IPV6
Router(config-fmm)# record ipv6 extended
Router(config-fmm)#exporter DetectPro_GPB
Router(config-fmm)# cache entries 1000000
Router(config-fmm)#cache entries active 1
Router(config-fmm)#cache entries inactive 1
Router(config-fmm)#cache timeout inactive 1
Router(config-fmm)#cache timeout rate-limit 1000000
Router(config-fmm)#exit
Router(config)#flow monitor-map DetectPro_Monitor_IPV4
Router(config-fmm)# record ipv4 extended
Router(config-fmm)#exporter DetectPro_GPB
Router(config-fmm)# cache entries 1000000
Router(config-fmm)#cache entries active 1
Router(config-fmm)#cache entries inactive 1
Router(config-fmm)#cache timeout inactive 1
Router(config-fmm)#cache timeout rate-limit 1000000
Router(config-fmm)#exit
//Configuring Monitor Map MPLS
Router(config)#flow monitor-map mon_mpls_ipv4_ipv6
Router(config-fmm)#record mpls ipv4-ipv6-fields
Router(config-fmm)#exporter DetectPro_MPLS

```

```

Router(config-fmm)#cache entries 1000000
Router(config-fmm)#cache timeout active 1
Router(config-fmm)#cache timeout inactive 1
Router(config-fmm)#cache timeout rate-limit 1000000
Router(config-fmm)#exit
//Configuring Exporter Map
Router(config)#flow exporter-map DetectPro_GPB
Router(config-fem)#version protobuf
Router(config-fem)#transport udp 5005
Router(config-fem)#source TenGigE0/0/0/16
Router(config-fem)#destination 15.1.1.2
Router(config-fem)#exit
//Configuring Exporter Map MPLS
Router(config)#flow exporter-map DetectPro_MPLS
Router(config-fem)#version protobuf
Router(config-fem)#transport udp 5005
Router(config-fem)#source loopback101
Router(config-fem)#destination 15.1.1.2
Router(config-fem)#exit
//Configuring Sampler Map
Router(config)#sampler-map DetectPro_NFv9
Router(config-sm)#random 1 out-of 100
!
//Configuring Sampler Map samp_MPLS
Router(config-sm)#random 1 out-of 100
!
//Configuring Interface MPLS
Router(config)#interface TenGigE0/0/0/8
Router(config-if)#ipv4 address 7.7.1.1 255.255.0.0
Router(config-if)#flow mpls monitor mon_mpls_ipv4_ipv6 sampler samp_mpls ingress
!

```

For more information on the DDoS edge protection, see [Cisco Secure DDoS Edge Protection Data Sheet](#).

Verify DDoS edge protection application configuration

To ensure the DDoS controller has applied the configuration to the device, check the active configuration on the router.

1. Execute the **show running-config appmgr** command on the router to verify the appmgr configuration.

```

RP/0/RP0/CPU0:Router#show running-config appmgr
Thu Jun  1 07:33:36.741 UTC
appmgr
  application esentryd
    activate type docker source esentryd-cisco-20230431633 docker-run-opts "--env-file
    /harddisk:/ENV_6478443711ac6830700dlaeb --net=host"
  !
!

```

2. Execute the **show flow monitor** command on the router to check the monitor map that is automatically created.

```

Router#show flow monitor DetectPro_Monitor_IPV4 cache location 0/0/CPU0
Cache summary for Flow Monitor DetectPro_Monitor_IPV4:
Cache size:                               1000000
Current entries:                           0
Flows added:                               2243884200
Flows not added:                           0
Ager Polls:                               2243884200
  - Active timeout                         0
  - Inactive timeout                       0

```

```

- Immediate                                0
- TCP FIN flag                             0
- Emergency aged                           0
- Counter wrap aged                         0
- Total                                   2243884200
Periodic export:
- Counter wrap                             0
- TCP FIN flag                             0
Flows exported                             2243884200

```

```

Matching entries:                          0
!

```

Router#**show flow monitor DetectPro_Monitor_IPV6 cache location 0/0/CPU0**

Cache summary for Flow Monitor DetectPro_Monitor_IPV6:

```

Cache size:                               1000000
Current entries:                           0
Flows added:                               59971
Flows not added:                           0
Ager Polls:                               94437
- Active timeout                           59971
- Inactive timeout                         0
- Immediate                               0
- TCP FIN flag                             0
- Emergency aged                           0
- Counter wrap aged                         0
- Total                                   59971
Periodic export:
- Counter wrap                             0
- TCP FIN flag                             0
Flows exported                             59971

```

```

Matching entries:                          0

```

Router#**show flow monitor mon_mpls_ipv4_ipv6 cache format record location 0/RP0/CPU0**

Cache summary for Flow Monitor mon_mpls_ipv4_ipv6:

```

Cache size:                               1000000
Current entries:                           0
Flows added:                               963
Flows not added:                           0
Ager Polls:                               83
- Active timeout                           963
- Inactive timeout                         0
- Immediate                               0
- TCP FIN flag                             0
- Emergency aged                           0
- Counter wrap aged                         0
- Total                                   963
Periodic export:
- Counter wrap                             0
- TCP FIN flag                             0
Flows exported                             963
!

```

3. Execute the **show flow exporter** command on the router to check the exporter map that is automatically created.

```

RP/0/RP0/CPU0:Router#show flow exporter
exporter exporter-map
RP/0/RP0/CPU0:tortin#show flow exporter DetectPro_GPB location 0/0/CPU0
Thu Nov 16 06:13:58.059 UTC
Flow Exporter: DetectPro_GPB
Export Protocol: protobuf

```

```

Flow Exporter memory usage: 5265344
Used by flow monitors: DetectPro_Monitor_IPV4
                      DetectPro_Monitor_IPV6

Status: Disabled
Transport:  UDP
Destination: 15.1.1.2      (5005) VRF default
Source:      0.0.0.0      (54482)
Flows exported:                                0 (0 bytes)
Flows dropped:                                0 (0 bytes)

Templates exported:                            0 (0 bytes)
Templates dropped:                            0 (0 bytes)

Option data exported:                         0 (0 bytes)
Option data dropped:                         0 (0 bytes)

Option templates exported:                    0 (0 bytes)
Option templates dropped:                    0 (0 bytes)

Packets exported:                            20355756 (27716506821 bytes)
Packets dropped:                             0 (0 bytes)

Total export over last interval of:
  1 hour:                                     12 pkts
                                              1879 bytes
                                              12 flows
  1 minute:                                  0 pkts
                                              0 bytes
                                              0 flows
  1 second:                                 0 pkts
                                              0 bytes
                                              0 flows

Router#show flow exporter
exporter exporter-map
Router#show flow exporter exp_mpls_ipv4_ipv6 location 0/RP0/CPU0
Flow Exporter: exp_mpls_ipv4_ipv6
Export Protocol: protobuf
Flow Exporter memory usage: 5265480
Used by flow monitors: mon_mpls_ipv4_ipv6

Status: Normal
Transport:  UDP
Destination: 15.1.1.2      (5005) VRF default
Source:      17.1.1.1      (54341)
Flows exported:                                963 (159519 bytes)
Flows dropped:                                0 (0 bytes)

Templates exported:                            0 (0 bytes)
Templates dropped:                            0 (0 bytes)

Option data exported:                         0 (0 bytes)
Option data dropped:                         0 (0 bytes)

Option templates exported:                    0 (0 bytes)
Option templates dropped:                    0 (0 bytes)

Packets exported:                            122 (159519 bytes)
Packets dropped:                             0 (0 bytes)

Total export over last interval of:
  1 hour:                                     92 pkts
                                              119950 bytes
                                              727 flows

```

```
1 minute:                122 pkts
                        159519 bytes
                        963 flows
1 second:                 0 pkts
                        0 bytes
                        0 flows
```

4. Execute the **show appmgr application-table** command on the router to check the status of docker application.

```
RP/0/RP0/CPU0:Router#show appmgr application-table
```

```
Thu Nov 16 06:13:58.059 UTC
```

```
Name      Type    Config State Status
```

```
-----  
esentryd Docker Activated Up 8 minutes
```

```
RP/0/RP0/CPU0:Router#
```




APPENDIX **A**

Packet I/O Functionality and Hosting Applications

- [Setting up Application Hosting Environment, on page 35](#)
- [Hosting an Application in Docker Containers, on page 52](#)
- [Boot Devices Using PXE Server Running in a Docker Container, on page 57](#)
- [Hosting and Activating the PXE Server Docker on Cisco 8201 Router using Application Manager, on page 60](#)
- [CPU-Based Packet Generator, on page 61](#)

Setting up Application Hosting Environment

This section illustrates how, with the Packet I/O functionality, you can use Linux applications to manage communication with the IOS XR interfaces. It describes how the OS environment must be set up to establish packet I/O communication with hosted applications.

Verify Reachability of IOS XR and Packet I/O Infrastructure

Feature Name	Release Information	Description
Virtual IP address in the Linux networking stack	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*) *This feature is supported on: • 8712-MOD-M • 8011-4G24Y4H-I

Feature Name	Release Information	Description
Virtual IP address in the Linux networking stack	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is supported on: • 8212-48FH-M • 8711-32FH-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
Virtual IP address in the Linux networking stack	Release 7.5.2	Virtual IP addresses allow a single IP address to connect to the current active RP after an RP switchover event. In addition, this functionality enables your network stack to support virtual IP addresses for third-party applications and IOS XR applications that use the Linux networking stack. The following commands are modified: • ipv4 virtual address • ipv6 virtual address • show linux networking interfaces address-only
Automatic Synchronization of Secondary IPv4 addresses from XR to Linux OS	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100])(select variants only*) *This feature is supported on: • 8712-MOD-M • 8011-4G24Y4H-I

Feature Name	Release Information	Description
Automatic Synchronization of Secondary IPv4 addresses from XR to Linux OS	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100])(select variants only*); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is supported on: • 8212-48FH-M • 8711-32FH-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
Automatic Synchronization of Secondary IPv4 addresses from XR to Linux OS	Release 7.5.3	Now the configured interface secondary IPv4 addresses on the Cisco IOS XR software are automatically synchronized to Linux operating system. The third-party applications on Cisco IOS XR can use the secondary IPv4 addresses without any manual intervention. Earlier, you had to configure the secondary IPv4 addresses on the Linux operating system manually.
Automatic Synchronization of Secondary IPv6 addresses from XR to Linux OS	Release 7.11.1	Now the configured interface secondary IPv6 addresses on the Cisco IOS XR software are automatically synchronized to Linux operating system. The third-party applications on Cisco IOS XR can use the secondary IPv6 addresses without any manual intervention. Earlier, you had to configure the secondary IPv6 addresses on the Linux operating system manually.

Interfaces configured on IOS XR are programmed into the Linux kernel. These interfaces allow Linux applications to run as if they were running on a regular Linux system. This packet I/O capability ensures that off-the-shelf Linux applications can be run alongside IOS XR, allowing operators to use their existing tools and automate deployments with IOS XR.

The IP address on the Linux interfaces, MTU settings, MAC address are inherited from the corresponding settings of the IOS XR interface. Accessing the global VRF network namespace ensures that when you issue the **bash** command, the default or the global VRF in IOS XR is reflected in the kernel. This ensures default reachability based on the routing capabilities of IOS XR and the packet I/O infrastructure.

Virtual addresses can be configured to access a router from the management network such as gRPC using a single virtual IP address. On a device with two or more RPs, the virtual address refers to the management interface that is currently active. This functionality can be used across RP failover without the information of which RP is currently active. This is applicable to the Linux packet path.

Automatic Synchronization of Secondary IPv4 and IPv6 addresses from XR to Linux OS

The secondary IPv4 and IPv6 addresses that are configured for an XR interface are now synchronized into the Linux operating system automatically. With this secondary IPv4 and IPv6 address synchronization, the third party applications that are deployed on Cisco IOS XR can now use the secondary addresses. Prior to this release, only primary IPv4 and IPv6 addresses were supported and the secondary IPv4 and IPv6 addresses had to be configured manually in the Linux operating system.

Exposed XR interfaces (EXIs) and address-only interfaces support secondary IPv4 and IPv6 address synchronization:

- EXIs have secondary IP addresses added to their corresponding Linux interface
- Address-only interfaces have secondary IP addresses added to the Linux loopback device. For additional information on address-only interfaces, see [show linux networking interfaces address-only](#).

The restrictions of secondary IPv4 addresses synchronization are:

- Secondary IPv4 addresses are not synchronized from Linux to XR for Linux-managed interfaces.
- The **ifconfig** Linux command only displays the first configured IPv4 address. To view the complete list of IPv4 addresses, use the **ip addr show** Linux command.

For additional information on secondary IPv4 addresses, see [ipv4 address \(network\)](#) and [ipv6 address](#).

You can run **bash** commands at the IOS XR router prompt to view the interfaces and IP addresses stored in global VRF. When you access the Cisco IOS XR Linux shell, you directly enter the global VRF.

Procedure

Step 1 From your Linux box, access the IOS XR console through SSH, and log in.

Example:

```
cisco@host:~$ ssh root@192.168.122.188
root@192.168.122.188's password:
Router#
```

Step 2 View the ethernet interfaces on IOS XR.

Example:

```
Router#show ip interface brief
Interface IP-Address Status Protocol Vrf-Name
FourHundredGigE0/0/0/0 unassigned Shutdown Down default
FourHundredGigE0/0/0/1 unassigned Shutdown Down default
FourHundredGigE0/0/0/2 unassigned Shutdown Down default
FourHundredGigE0/0/0/3 unassigned Shutdown Down default
```

```

FourHundredGigE0/0/0/4 unassigned Shutdown Down default
FourHundredGigE0/0/0/5 unassigned Shutdown Down default
FourHundredGigE0/0/0/6 unassigned Shutdown Down default
FourHundredGigE0/0/0/7 unassigned Shutdown Down default
FourHundredGigE0/0/0/8 unassigned Shutdown Down default
FourHundredGigE0/0/0/9 unassigned Shutdown Down default
FourHundredGigE0/0/0/10 unassigned Shutdown Down default
FourHundredGigE0/0/0/11 unassigned Shutdown Down default
FourHundredGigE0/0/0/12 unassigned Shutdown Down default
FourHundredGigE0/0/0/13 unassigned Shutdown Down default
FourHundredGigE0/0/0/14 unassigned Shutdown Down default
FourHundredGigE0/0/0/15 unassigned Shutdown Down default
FourHundredGigE0/0/0/16 unassigned Shutdown Down default
FourHundredGigE0/0/0/17 unassigned Shutdown Down default
FourHundredGigE0/0/0/18 unassigned Shutdown Down default
FourHundredGigE0/0/0/19 unassigned Shutdown Down default
FourHundredGigE0/0/0/20 unassigned Shutdown Down default
FourHundredGigE0/0/0/21 unassigned Shutdown Down default
FourHundredGigE0/0/0/22 unassigned Shutdown Down default
FourHundredGigE0/0/0/23 unassigned Shutdown Down default
HundredGigE0/0/0/24 10.1.1.10 Up Up default
HundredGigE0/0/0/25 unassigned Shutdown Down default
HundredGigE0/0/0/26 unassigned Shutdown Down default
HundredGigE0/0/0/27 unassigned Shutdown Down default
HundredGigE0/0/0/28 unassigned Shutdown Down default
HundredGigE0/0/0/29 unassigned Shutdown Down default
HundredGigE0/0/0/30 unassigned Shutdown Down default
HundredGigE0/0/0/31 unassigned Shutdown Down default
HundredGigE0/0/0/32 unassigned Shutdown Down default
HundredGigE0/0/0/33 unassigned Shutdown Down default
HundredGigE0/0/0/34 unassigned Shutdown Down default
HundredGigE0/0/0/35 unassigned Shutdown Down default
MgmtEth0/RP0/CPU0/0 192.168.122.22 Up Up default

```

Note

Use the **ip addr show** or **ip link show** commands to view all corresponding interfaces in Linux. The IOS XR interfaces that are admin-down state also reflects a Down state in the Linux kernel.

Step 3

Check the IP and MAC addresses of the interface that is in Up state. Here, interfaces HundredGigE0/0/0/24 and MgmtEth0/RP0/CPU0/0 are in the Up state.

Example:

```

Router#show interfaces HundredGigE0/0/0/24
...
HundredGigE0/0/0/24 is up, line protocol is up
Interface state transitions: 4
Hardware is HundredGigE0/0/0/24, address is 5246.e8a3.3754 (bia
5246.e8a3.3754)
Internet address is 10.1.1.1/24
MTU 1514 bytes, BW 1000000 Kbit (Max: 1000000 Kbit)
reliability 255/255, txload 0/255, rxload 0/255
Encapsulation ARPA,
Duplex unknown, 1000Mb/s, link type is force-up
output flow control is off, input flow control is off
loopback not set,
Last link flapped 01:03:50
ARP type ARPA, ARP timeout 04:00:00
Last input 00:38:45, output 00:38:45
Last clearing of "show interface" counters never
5 minute input rate 0 bits/sec, 0 packets/sec
5 minute output rate 0 bits/sec, 0 packets/sec
12 packets input, 1260 bytes, 0 total input drops
0 drops for unrecognized upper-level protocol

```

```
Received 2 broadcast packets, 0 multicast packets
0 runs, 0 giants, 0 throttles, 0 parity
0 input errors, 0 CRC, 0 frame, 0 overrun, 0 ignored, 0 abort
12 packets output, 1224 bytes, 0 total output drops
Output 1 broadcast packets, 0 multicast packets
```

Step 4 Verify that the bash command runs in global VRF to view the network interfaces.

Example:

```
Router#bash -c ifconfig
Hu0_0_0_24 Link encap:Ethernet HWaddr 78:e7:e8:d3:20:c0
inet addr:10.1.1.10 Bcast:0.0.0.0 Mask:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:4 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:360 (360.0 B) TX bytes:0 (0.0 B)
Mg0_RP0_CPU0_0 Link encap:Ethernet HWaddr 54:00:00:00:bd:49
inet addr:192.168.122.22 Bcast:0.0.0.0 Mask:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:3859 errors:0 dropped:0 overruns:0 frame:0
TX packets:1973 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:2377782 (2.2 MiB) TX bytes:593602 (579.6 KiB)
lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:242 errors:0 dropped:0 overruns:0 frame:0
TX packets:242 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:12100 (11.8 KiB) TX bytes:12100 (11.8 KiB)
to_xr Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:1 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:500
RX bytes:0 (0.0 B) TX bytes:60 (60.0 B)
```

The `to_xr` interface indicates access to the global VRF.

Step 5 Access the Linux shell.

Example:

```
Router#bash
[ios:~]$
```

Step 6 (Optional) View the IP routes used by the `to_xr` interfaces.

Example:

```
[ios:~]$ip route
default dev to_xr scope link metric 2048
6.1.0.0/16dev Mg0_RP0_CPU0_0 proto kernel scope link src 6.1.22.41
20.1.0.0/16dev Hu0_0_0_0 proto kernel scope link src 20.1.1.1
20.2.0.0/16dev Hu0_0_0_20 proto kernel scope link src 20.2.1.1
30.1.0.0/24dev BE500 proto kernel scope link src 30.1.0.1
172.17.0.0/16dev docker0 proto kernel scope link src 172.17.0.1linkdown
```

Note

You can also enter the global VRF directly after logging into IOS XR using the **run ip netns exec vrf-default bash** command.

Programme Routes in the Kernel

The basic routes required to allow applications to send or receive traffic can be programmed into the kernel. The Linux network stack that is part of the kernel is used by normal Linux applications to send/receive packets. In an IOS XR stack, IOS XR acts as the network stack for the system. Therefore to allow the Linux network stack to connect into and use the IOS XR network stack, basic routes must be programmed into the Linux Kernel.

Procedure

Step 1 View the routes from the bash shell.

Example:

```
[ios:~]$ip route
default dev to_xr scope link src 10.1.1.10 metric 2048
10.1.1.0/24 dev Hu0_0_0_24 proto kernel scope link src 10.1.1.10
192.168.122.0/24 dev Mg0_RP0_CPU0_0 proto kernel scope link src 192.168.122.22
```

Step 2 Programme the routes in the kernel.

Two types of routes can be programmed in the kernel:

- **Default Route:** The default route sends traffic destined to unknown subnets out of the kernel using a special `to_xr` interface. This interface sends packets to IOS XR for routing using the routing state in XR Routing Information Base (RIB) or Forwarding Information Base (FIB). The `to_xr` interface does not have an associated IP address. In Linux, most applications expect the outgoing packets to use the IP address of the outgoing interface as the source IP address.

With the `to_xr` interface, because there is no IP address, a source hint is required. The source hint can be changed to use the IP address another physical interface IP or loopback IP address. In the following example, the source hint is set to 10.1.1.10, which is the IP address of the `Hu0_0_0_24` interface. To use the Management port IP address, change the source hint:

Router#**bash**

```
[ios:~]$ip route replace default dev to_xr scope link src 192.168.122.22 metric 2048
```

```
[ios:~]$ip route
default dev to_xr scope link src 192.168.122.22 metric 2048
10.1.1.0/24 dev Hu0_0_0_24 proto kernel scope link src 10.1.1.10
192.168.122.0/24 dev Mg0_RP0_CPU0_0 proto kernel scope link src 192.168.122.22
```

With this updated source hint, any default traffic exiting the system uses the Management port IP address as the source IP address.

- **Local or Connected Routes:** The routes are associated with the subnet configured on interfaces. For example, the 10.1.1.0/24 network is associated with the `Hu0_0_0_24` interface, and the 192.168.122.0/24 subnet is associated with the `Mg0_RP0_CPU0` interface.

Configure VRFs in the Kernel

VRFs configured in IOS XR are automatically synchronized to the kernel. In the kernel, the VRFs appear as network namespaces (netns). For every globally-configured VRF, a Linux network namespace is created. With this capability it is possible to isolate Linux applications or processes into specific VRFs like an out-of-band management VRF and open-up sockets or send or receive traffic only on interfaces in that VRF.

Every VRF, when synchronized with the Linux kernel, is programmed as a network namespace with the same name as a VRF but with the string `vrf` prefixed to it. The default VRF in IOS XR has the name `default`. This name gets programmed as `vrf-default` in the Linux kernel.

The following example shows how to configure a custom VRF `blue`:

Procedure

Step 1 Identify the current network namespace or VRF.

Example:

```
[ios:~]$ip netns identify $$
vrf-default
global-vrf
```

Step 2 Configure a custom VRF `blue`.

Example:

```
Router#conf t

Router(config)#vrf blue
Router(config-vrf)#commit
```

Step 3 Verify that the VRF `blue` is configured in IOS XR.

Example:

```
Router#show run vrf
vrf blue
!
```

Step 4 Verify that the VRF `blue` is created in the kernel.

Example:

```
Router#bash

[ios:~]$ls -l /var/run/netns
total 0
-r--r--r--. 1 root root 0 Jul 30 04:17 default
-r--r--r--. 1 root root 0 Jul 30 04:17 global-vrf
-r--r--r--. 1 root root 0 Jul 30 04:17 tpnns
-r--r--r--. 1 root root 0 Aug 1 17:01 vrf-blue
```

```
-r--r--r--. 1 root root 0 Jul 30 04:17 vrf-default
-r--r--r--. 1 root root 0 Jul 30 04:17 xrnns
```

Step 5 Access VRF `blue` to launch and execute processes from the new network namespace.

Example:

```
[ios:~]$ip netns exec vrf-blue bash
[ios:~]$
[ios:~]$ip netns identify $$
vrf-blue
[ios:~]$
```

Running an `ifconfig` command shows only the default `to-xr` interface because there is no IOS XR interface in this VRF.

```
[ios:~]$ifconfig
lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
to_xr Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:500
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
[ios:~]$
```

Step 6 Configure an interface in the VRF `blue` in IOS XR. This interface will be configured automatically in the network namespace `vrf-blue` in the kernel.

Example:

The following example shows how to configure HundredGigE 0/0/0/24 interface in `vrf-blue` from IOS XR:

```
Router#conf t
Router(config)#int HundredGigE 0/0/0/24
Router(config-if)#no ipv4 address
Router(config-if)#vrf blue
Router(config-if)#ipv4 address 10.1.1.10/24
Router(config-if)#commit
```

Step 7 Verify that the HundredGigE 0/0/0/24 interface is configured in the VRF `blue` in IOS XR.

Example:

```
Router#show run int HundredGigE 0/0/0/24
interface HundredGigE0/0/0/24
vrf blue
ipv4 address 10.1.1.10 255.255.255.0
!
```

Step 8 Verify that the interface is configured in the VRF `blue` in the kernel.

Example:

```
Router#bash
Thu Aug 1 17:09:39.314 UTC
[ios:~]$
[ios:~]$ip netns exec vrf-blue bash
[ios:~]$
[ios:~]$ifconfig
```

```

Hu0_0_0_24 Link encap:Ethernet HWaddr 78:e7:e8:d3:20:c0
inet addr:10.1.1.10 Bcast:0.0.0.0 Mask:255.255.255.0
UP BROADCAST RUNNING MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
to_xr Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:500
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
[ios:~]$

```

Open Linux Sockets

The socket entries are programmed into the Local Packet Transport Services (LPTS) infrastructure that distributes the information through the line cards. Any packet received on a line card interface triggers an LPTS lookup to send the packet to the application opening the socket. Because the required interfaces and routes already appear in the kernel, the applications can open the sockets — TCP or UDP.

Procedure

Step 1 Verify that applications open up sockets.

Example:

```

Router#bash
[ios:~]$nc -l 0.0.0.0 -p 5000 &
[1] 1160
[ios:~]$
[ios:~]$netstat -nlp
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address Foreign Address State PID/Program name
tcp 0 0 0.0.0.0:5000 0.0.0.0:* LISTEN 1160/nc
tcp 0 0 0.0.0.0:57777 0.0.0.0:* LISTEN 14723/emsd
tcp 0 0 0.0.0.0:22 0.0.0.0:* LISTEN 8875/ssh_server
tcp6 0 0 :::22 :::* LISTEN 8875/ssh_server
udp 0 0 0.0.0.0:68 0.0.0.0:* 13235/xr_dhcpd
Active UNIX domain sockets (only servers)
Proto RefCnt Flags Type State I-Node PID/Program name Path
[ios:~]$exit
Logout
Router#
Router#show lpts pifib brief | i 5000
Thu Aug 1 17:16:00.938 UTC

```



```
IPv4 default TCP any 0/RP0/CPU0 any,5000 any
Router#
```

Step 2 Verify that the socket is open.

Example:

```
Router#show lpts pifib brief | i 5000
IPv4 default TCP any 0/RP0/CPU0 any,5000 any
```

Netcat starts listening on port 5000, which appears as an IPv4 TCP socket in the netstat output like a typical Linux kernel. This socket gets programmed to LPTS, creating a corresponding entry in the hardware to the lookup tcp port 5000. The incoming traffic is redirected to the kernel of the active RP where the netcat runs.

Send and Receive Traffic

Connect to the nc socket from an external server. For example, the nc socket was started in the `vrf-default` network namespace. So, connect over an interface that is in the same VRF.

```
[root@localhost ~]#nc -vz 192.168.122.22 5000
Ncat: Version 7.50 ( https://nmap.org/ncat )
Ncat: Connected to 192.168.122.22:5000.
Ncat: 0 bytes sent, 0 bytes received in 0.01 seconds.
```

Manage IOS XR Interfaces through Linux

The Linux system contains a number of individual network namespaces. Each namespace contains a set of interfaces that map to a single interface in the XR control plane. These interfaces represent the exposed XR interfaces (eXI). By default, all interfaces in IOS XR are managed through the IOS XR configuration (CLI or YANG models), and the attributes of the interface (IP address, MTU, and state) are inherited from the corresponding configuration and the state of the interface in XR.

With the new Packet I/O functionality, it is possible to have an IOS XR interface completely managed by Linux. This also means that one or more of the interfaces can be configured to be managed by Linux, and standard automation tools can be used on Linux servers can be used to manage interfaces in IOS XR.



Note Secondary IPv4 addresses cannot be managed by Linux.

Configure an Interface to be Linux-Managed

This section shows how to configure an interface to be Linux-managed.

Procedure

Step 1 Check the available exposed-interfaces in the system.

Example:

```
Router(config)#linux networking exposed-interfaces interface ?
BVI                Bridge-Group Virtual Interface
```

Bundle-Ether	Aggregated Ethernet interface(s) short name is BE
FiftyGigE	FiftyGigabitEthernet/IEEE 802.3 interface(s) short name is Fi
FortyGigE	FortyGigabitEthernet/IEEE 802.3 interface(s) short name is Fo
FourHundredGigE	FourHundredGigabitEthernet/IEEE 802.3 interface(s) short name is FH
GigabitEthernet	GigabitEthernet/IEEE 802.3 interface(s) short name is Gi
HundredGigE	HundredGigabitEthernet/IEEE 802.3 interface(s) short name is Hu
Loopback	Loopback interface(s) short name is Lo
MgmtEth	Ethernet/IEEE 802.3 interface(s) short name is Mg
TenGigE	TenGigabitEthernet/IEEE 802.3 interface(s) short name is Te
TwentyFiveGigE	TwentyFiveGigabitEthernet/IEEE 802.3 interface(s) short name is TF
TwoHundredGigE	TwoHundredGigabitEthernet/IEEE 802.3 interface(s) short name is TH

Step 2 Configure the interface to be managed by Linux.

Example:

The following example shows how to configure a HundredGigE interface to be managed by Linux:

```
Router#configure
Router(config)#linux networking exposed-interfaces interface HundredGigE 0/0/0/24 linux-managed
Router(config-exi-if)#commit
```

Example:

The following example shows how to configure a BVI5 interface to be managed by Linux:

```
Router#configure
Router(config)#linux networking exposed-interfaces interface BVI5 linux-managed
Router(config-exi-if)#commit
```

Step 3 View the interface details and the VRF.

Example:

The following example shows the information for HundredGigE interface:

```
Router#show run interface HundredGigE0/0/0/24
interface HundredGigE0/0/0/24
mtu 4110
vrf blue
ipv4 mtu 4096
ipv4 address 10.1.1.10 255.255.255.0
ipv6 mtu 4096
ipv6 address fe80::7ae7:e8ff:fed3:20c0 link-local
!
```

Example:

The following example shows the information for BVI5 interface:

```
Router#show run interface bvi5
interface bvi5
mtu 1514
ipv4 mtu 1500
ipv4 address 90.9.9.9 255.255.255.0
ipv6 mtu 1500
ipv6 address fe80::4ee1:75ff:fe74:a80c link-local
!
```

Step 4 Verify the configuration in XR.

Example:

The following example shows the configuration for HundredGigE interface:

```
Router#show running-config linux networking
```

```

linux networking
  exposed-interfaces
    interface HundredGigE0/0/0/24 linux-managed
  !
!
!

```

Example:

The following example shows the configuration for BVI5 interface:

```
Router#show running-config linux networking
```

```

linux networking
  exposed-interfaces
    interface BVI5 linux-managed
  !
!
!

```

Step 5 Verify the configuration from Linux.**Example:**

The following example shows the configuration for HundredGigE interface:

```
Router#bash
Router:Aug 1 17:40:02.873 UTC: bash_cmd[67805]: %INFRA-INFRA_MSG-5-RUN_LOGIN : User vagrant logged
into shell from vty0
```

```
[ios:~]$ip netns exec vrf-blue bash
```

```

[ios:~]$ifconfig
lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
to_xr Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:500
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)

```

```

[ios:~]$ifconfig -a
Hu0_0_0_24 Link encap:Ethernet HWaddr 78:e7:e8:d3:20:c0
BROADCAST MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
to_xr Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1

```

Configure New IP address on the Interface in Linux

```
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:500
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

Example:

The following example shows the configuration for BVI5 interface:

```
Router#bash
Router:Aug 1 17:40:02.873 UTC: bash_cmd[67805]: %INFRA-INFRA_MSG-5-RUN_LOGIN : User vagrant logged
into shell from vty0

[ios:~]$ifconfig BVI5
lo Link encap:Local LoopbackBVI5 Link encap:Ethernet HWaddr 4c:e1:75:74:a8:0c
BROADCAST MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

Configure New IP address on the Interface in Linux

This section shows how to configure a new IP address on the Linux-managed interface.

Procedure

Step 1 Configure the IP address on the interface.

Example:

```
[ios:~]$ip addr add 10.1.1.10/24 dev Hu0_0_0_24
[ios:~]$Router:Aug 1 17:41:11.546 UTC: xlncd[253]: %MGBL-CONFIG-6-DB_COMMIT : Configuration
committed by user 'system'. Use 'show configuration commit changes 1000000021' to view the changes.
```

Step 2 Verify that the new IP address is configured.

Example:

```
[ios:~]$ifconfig Hu0_0_0_24
Hu0_0_0_24 Link encap:Ethernet HWaddr 78:e7:e8:d3:20:c0
inet addr:10.1.1.10 Bcast:0.0.0.0 Mask:255.255.255.0
BROADCAST MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

Configure Custom MTU Setting

This section shows how to bring up the interface and configure a custom MTU in a Linux-managed interface.

Procedure

Step 1 Configure the MTU setting.

Example:

```
[ios:~]$ifconfig Hu0_0_0_24 up

[ios:~]$Router:Aug 1 17:41:54.824 UTC: ifmgr[266]: %PKT_INFRA-LINK-3-UPDOWN : Interface
HundredGigE0/0/0/24, changed state to Down
Router:Aug 1 17:41:54.824 UTC: ifmgr[266]: %PKT_INFRA-LINEPROTO-5-UPDOWN : Line protocol on
Interface HundredGigE0/0/0/24, changed state to Down
Router:Aug 1 17:41:56.448 UTC: xlncd[253]: %MGBL-CONFIG-6-DB_COMMIT : Configuration committed by
user 'system'. Use 'show configuration commit changes 1000000022' to view the changes.
Router:Aug 1 17:41:56.471 UTC: ifmgr[266]: %PKT_INFRA-LINK-3-UPDOWN : Interface
HundredGigE0/0/0/24, changed state to Up
Router:Aug 1 17:41:56.484 UTC: ifmgr[266]: %PKT_INFRA-LINEPROTO-5-UPDOWN : Line protocol on
Interface HundredGigE0/0/0/24, changed state to Up
Router:Aug 1 17:41:58.493 UTC: xlncd[253]: %MGBL-CONFIG-6-DB_COMMIT : Configuration committed by
user 'system'. Use 'show configuration commit changes 1000000023' to view the changes.

[ios:~]$
[ios:~]$ ip link set dev Hu0_0_0_24 mtu 4096
[ios:~]$
[ios:~]$Router:Aug 1 17:42:46.830 UTC: xlncd[253]: %MGBL-CONFIG-6-DB_COMMIT : Configuration
committed by user 'system'. Use 'show configuration commit changes 1000000024' to view the changes.
```

Step 2 Verify that the MTU setting has been updated in Linux.

Example:

```
[ios:~]$ifconfig
Hu0_0_0_24 Link encap:Ethernet HWaddr 78:e7:e8:d3:20:c0
inet addr:10.1.1.10 Bcast:0.0.0.0 Mask:255.255.255.0
inet6 addr: fe80::7ae7:e8ff:fed3:20c0/64 Scope:Link
UP BROADCAST RUNNING MULTICAST MTU:4096 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:8 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1000
RX bytes:0 (0.0 B) TX bytes:648 (648.0 B)
lo Link encap:Local Loopback
inet addr:127.0.0.1 Mask:255.0.0.0
inet6 addr: ::1/128 Scope:Host
UP LOOPBACK RUNNING MTU:65536 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:1
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
to_xr Link encap:UNSPEC HWaddr 00-00-00-00-00-00-00-00-00-00-00-00-00-00-00-00
UP POINTOPOINT RUNNING NOARP MULTICAST MTU:1500 Metric:1
RX packets:0 errors:0 dropped:0 overruns:0 frame:0
TX packets:0 errors:0 dropped:0 overruns:0 carrier:0
collisions:0 txqueuelen:500
RX bytes:0 (0.0 B) TX bytes:0 (0.0 B)
```

Step 3 Check the effect on the IOS XR configuration with the change in MTU setting on this interface.

Example:

```
Router#show running-config int HundredGigE0/0/0/24
interface HundredGigE0/0/0/24
  mtu 4110
```

```

vrf blue
ipv4 mtu 4096
ipv4 address 10.1.1.10 255.255.255.0
ipv6 mtu 4096
ipv6 address fe80::7ae7:e8ff:fed3:20c0 link-local
!
!
Router#
Router#show ip int br | i HundredGigE0/0/0/24
HundredGigE0/0/0/24 10.1.1.10 Up Up blue

```

The output indicates that the interface acts as a regular Linux interface, and IOS XR configuration receives inputs from Linux.

Configure Traffic Protection for Linux Networking

Traffic protection provides a mechanism to configure Linux firewalls using IOS XR configuration. These rules can be used to restrict traffic to Linux applications. You can restrict traffic to Linux applications using native Linux firewalls or configuring IOS XR Linux traffic protection. It is not recommended to use both mechanisms at the same time. Any combination of remote address, local address and ingress interface can be specified as rules to either allow or deny traffic. However, at least one parameter must be specified for the traffic protection rule to be valid.



Note If traffic is received on a protocol or port combination that has no traffic protection rules configured, then all traffic is allowed by default.

This example explains how to configure a traffic protection rule on IOS XR to deny all traffic on port 999 except for traffic arriving on interface HundredGigE0/0/0/25.

Procedure

Step 1 Configure traffic protection rules.

Example:

```

Router(config)#linux networking vrf default address-family ipv4 protection protocol
tcp local-port 999 default-action deny permit hundredgigE0/0/0/25
Router(config)#commit

```

where —

- **address-family:** Configuration for a particular IPv4 or IPv6 address family.
- **protection:** Configure traffic protection for Linux networking.
- **protocol:** Select the supported protocol - TCP or UDP.
- **local-port:** L4 port number to specify traffic protection rules for Linux networking.
- **port number:** Port number ranges from 1 to 65535 or all ports.

- **default-action:** Default action to take for packets matching this traffic protection service.
- **deny:** Drop packets for this service.
- **permit:** Permit packets to reach Linux application for this service.

Step 2 Verify that the traffic protection rule is applied successfully.

Example:

```
Router(config)#show run linux networking
linux networking
vrf default
address-family ipv4
protection
protocol tcp local-port 999 default-action deny
permit interface HundredGigE0/0/0/25
!
!
!
```

Synchronize Statistics Between IOS XR and Linux

This example shows how the bundle-ether interface packet statistics are synchronized between IOS XR and Linux. The packet and byte counters maintained by Linux for IOS XR interfaces display only the traffic sourced in Linux. You can configure to periodically synchronize these counters with the IOS XR statistics for the interfaces.

Procedure

Step 1 Configure the statistics synchronization including the direction and synchronization interval.

Example:

The following example shows statistics synchronization in global configuration:

```
Router(config)#linux networking statistics-synchronization from-xr
every 30s
```

Example:

The following example shows statistics synchronization in exposed-interface configuration:

```
Router(config)#linux networking exposed-interfaces interface
bundle-ether 1 statistics-synchronization from-xr every 10s
```

where —

- **from-xr:** The direction indicating that the interface packet statistics will be pushed from IOS XR to the Linux kernel.
- **every:** Shows the frequency at which to synchronize statistics. The intervals supported for global configuration are 30s and 60s. The intervals supported for exposed interfaces are 5s, 10s, 30s or 60s. The interval *s* is in seconds.

Step 2 Verify that the statistics synchronization is applied successfully on IOS XR.

Example:

```
Router#show run linux networking
linux networking
vrf default
address-family ipv4
protection
protocol tcp local-port all default-action deny
permit interface bundle-ether 1
!
!
!
!
exposed-interfaces
interface bundle-ether 1 linux-managed
statistics-synchronization from-xr every 10s
!
!
!
```

For troubleshooting purposes, use the **show tech-support linux networking** command to display debugging information.

Hosting an Application in Docker Containers

This section provides the procedure for hosting an application in docker containers.

The iPerf application is used as an example to demonstrate the hosting at a server and a client router.

[Verify Reachability of IOS XR and Packet I/O Infrastructure, on page 35](#) on the router that hosts the iPerf application. You can enable the following Packet I/O functionalities on the server and client routers prior to hosting the iPerf application in docker containers, for additional features on the routers:

- **Program Routes in the Kernel**—to send or receive traffic to a remote network using a specific interface.
- **Configure VRFs in the Kernel**—to run the iperf application in a non-default VRF.
- **Configure Traffic Protection for Linux Networking**—to secure the router by restricting access to the router on which the iperf application is hosted.

You build the docker image of the application following the standard [docker build](#) procedures. The docker image of any application (for example, iPerf) is built only once, after which, that docker image can be copied to other devices where the application can be hosted in docker containers.

Docker Operations

This section describes basic docker operations and the commands required for hosting and maintaining the applications:

Commands for Hosting Applications

- **Pull or Load the image:** This function copies a docker image to a device.
 - Pull—

The following command pulls the Docker image from a local docker registry. Ensure that the registry is accessible from the router.

```
[ios:~]$docker pull ufi-lnx:5001/alpine
❑! Here ufi-lnx is the docker registry reachable through 10.105.39.169!-->
Using default tag: latest
latest: Pulling from library/alpine
c9b1b535fdd9: Pull complete
Digest: sha256:ab00606a42621fb68f2ed6ad3c88be54397f981a7b70a79db3d1172b11c4367d
Status: Downloaded newer image for alpine:latest
[ios:~]$
```

Instead of being pulled from the registry, docker images can be loaded from images saved as tar files.

• Load—

You save the docker image as a tar file on the build host using the **docker save -o <path for generated tar file> <image name>** command. You copy the tar file into the target router, using the following command:

You copy the tar file into the target router, using the following command:

```
Router#scp root@10.105.227.122:/var/www/html/alpine.tar
Tue Mar 10 02:42:38.598 UTC
Connecting to 10.105.227.122...
Password:
Transferred 639972864 Bytes
639972864 bytes copied in 55 sec (11606958)bytes/sec
Router#bash
Tue Mar 10 02:45:25.330 UTC
[ios:~]$docker load -i /tmp/alpine.tar
Loaded image: alpine:latest
ios:~]$docker images
REPOSITORY          TAG                IMAGE ID           CREATED
SIZE
alpine               latest            dc721c65d296      11 days ago
622MB
[ios:~]$
```

• View or List Docker Images

```
[ios:~]$docker images
REPOSITORY          TAG                IMAGE ID           CREATED           SIZE
alpine               latest            dc721c65d296      11 days ago      622MB
enipla               latest            fd31184c8c1b      6 weeks ago      581MB
[ios:~]$
```

• Run Container

```
[ios:~]$docker run -it alpine bash
root@alb719df1091:/#
root@alb719df1091:/#
root@alb719df1091:/#uname -a
Linux alb719df1091 4.8.28-WR9.0.0.20_cgl#1 SMP Wed Jan 8 11:16:16 UTC 2020 x86_64 x86_64
x86_64 GNU/Linux
root@alb719df1091:/#hostname
alb719df1091
root@alb719df1091:/#
[ios:~]$docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED           STATUS
PORTS              NAMES
alb719df1091       alpine             "bash"             About a minute ago Up
```

```
About a minute                                nifty_leavitt
[ios:~]$
```

• Attach to a Running Container

The **docker attach** command attaches the terminal to the running container and the **docker exec** command runs commands inside a working container.

```
[ios:~]$ docker attach docker1
#bash
root@e1e1924956df:/#
```

The **docker exec -it my_container_id sh** command executes a shell inside the container:

```
f3b-r1-pod9:/var/lib/docker/volumes]$ docker exec -it 57029028609a sh
#
```

• Stop Containers

Identify the container using the **docker ps** command and then stop the container using the **docker stop** command.

```
[ios:~]$ docker ps
CONTAINER ID        IMAGE               COMMAND             CREATED             STATUS
8782d4902312       alpine             "bash"             7 minutes ago      Up 6
minutes
9bd23408d640       alpine             "bash"             17 minutes ago     Up 17
minutes
[ios:~]$ docker stop 8782d4902312
```

• Stopping All Containers

```
[ios:~]$ docker stop $(docker ps -a -q)
```

Commands for Maintaining Containers

• Restart Policies

Docker restart policies start the containers automatically after the router reboots.

```
[ios:~]$ docker run -d --restart
```

• Clean up Containers and Images

You can clean images, containers, volumes, and networks that are dangling (and not associated with a container) by using the **docker system prune** command.

```
[ios:~]$ docker container prune
```

and

```
[ios:~]$ docker image prune
```

• Monitor Docker (View Docker Statistics)

You can view performance metrics, such as utilization of memory and CPU, and container-specific metrics, such as CPU limit and memory limit.

```
[ios:~]$ docker stats --no-stream
CONTAINER ID        NAME               CPU %               MEM USAGE / LIMIT   MEM
%                   NET I/O           BLOCK I/O          PIDS
alb719df1091       nifty_leavitt      0.00%              2.035MiB / 30.78GiB  0.01%
```

648B / 0B

0B / 0B

2

**Note**

For generic docker commands, see the Docker Release 18.05 documentation. (<https://docs.docker.com>)

Procedure for Hosting Applications in Docker Containers

1. Build the docker image following the standard docker build procedures. See [here](#).

The docker image is transferred to the IOS XR router (target router) using one of the following ways:

- The docker image is pulled from the docker image registry into the target router (or)
- The docker image is saved as the tar file in the build host and then the tar file is copied into the target router from the build host.

2. Start the docker container and run the application on the router.
3. Verify the hosted application in the docker container.

Run iPerf in Docker Container

As an example of application hosting in docker container, you can install iPerf client on Router A and check its connectivity with an iPerf server installed on Router B.

This figure illustrates the topology used in this example.

Figure 2: iPerf Hosted in a Docker Container



The following steps describe how to run the iPerf server and iPerf client applications on Router A and Router B.

Before you begin

Ensure that you have configured the two routers as shown in the figure-*iPerf application hosted in a Docker Container*.

Procedure

- Step 1** Copy the iPerf application tar file (for example, ubuntu-agnel-image.tar) on Router A.

Example:

```
Router#scp root@10.105.227.122:/var/www/html/ubuntu-agnel-27feb.tar $
Connecting to 10.105.227.122...
Password:
```

Verify the Application Hosted in the Docker Container

```
Transferred 639972864 Bytes
639972864 bytes copied in 55 sec (11606958)bytes/sec
```

Step 2 Load the docker instance on Router A by using the following command:

Example:

```
Router#bash
[ios:~]$docker load -i /tmp/ubuntu-agnel-27feb.tar
```

Step 3 View all docker images by using the following command:

Example:

```
[ios:~]$docker images ls
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
ubuntu-agnel-27feb	latest	dc721c65d296	11 days ago	622MB
ubuntu-agnel-slapi	latest	fd31184c8c1b	6 weeks ago	581MB

Step 4 Repeat Steps 1 through 3 on Router B.

Step 5 Configure the application to run as iPerf server on Router A.

Example:

```
[ios:~]$docker run -d -it -p 601:601 ubuntu-agnel-27feb sh
[ios:~]$iperf3 -s -B 172.17.0.2
```

Note

“-p 601:601 ubuntu-agnel-27feb sh” part of the command maps the Linux port with the docker instance port. 601 on the left hand side is the Docker instance mapping port and 601 on the right hand side is the Linux kernel port.

172.17.0.2 is the IP address of the server.

Step 6 Configure the application to run as iPerf client on Router B and establish connection to iPerf server on Router A.

Example:

```
[ios:~]$docker run -d -it -p 601:601 ubuntu-agnel-27feb sh
[ios:~]$iperf3 -c 172.17.0.2
```

Note

“-p 601:601 ubuntu-agnel-27feb sh” part of the command maps the Linux port with the docker instance port. 601 on the left hand side is the docker instance mapping port and 601 on the right hand side is the Linux kernel port.

172.17.0.2 is the IP address of the server.

Verify the Application Hosted in the Docker Container

To verify the applications hosted in the docker containers between Router A and Router B, use the **ping** command to check if the connection has been established between iPerf server and iPerf client.

Procedure

From the iPerf client on Router B, ping the iPerf server on Router A by providing the physical interface IP address to verify the connection between the iPerf server and client applications.

Example:

```
[ios:~]$ping 172.17.0.2
Type escape sequence to abort.
Sending 5, 100-byte ICMP Echos to 172.17.0.2, timeout is 2 seconds:
!!!!
Success rate is 100 percent (5/5), round-trip min/avg/max = 13/21/40 ms
```

172.17.0.2 is the IP address of the client.

Boot Devices Using PXE Server Running in a Docker Container

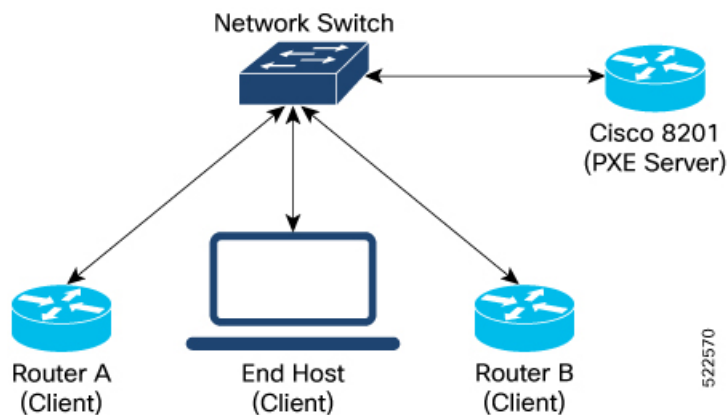
Feature Name	Release Information	Description
Boot Devices Using PXE Server Running in a Docker Container	Release 25.1.1	Introduced in this release on: Fixed Systems (8700 [ASIC: K100], 8010 [ASIC: A100]) (select variants only*) *This feature is supported on Cisco 8712-MOD-M router.
Boot Devices Using PXE Server Running in a Docker Container	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100]); Modular Systems (8800 [LC ASIC: P100]) (select variants only*) *This feature is supported on: • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM • 8212-48FH-M • 8711-32FH-M
Boot Devices Using PXE Server Running in a Docker Container	Release 24.2.1	Starting from Cisco IOS XR Release 24.2.1, the PXE server feature is deprecated and will not be supported in future releases. We recommend not to use this feature starting from Cisco IOS XR Release 24.2.1.

Feature Name	Release Information	Description
Boot Devices Using PXE Server Running in a Docker Container	Release 7.3.4	You can now boot your network devices with a PXE pre-boot execution environment (PXE or iPXE) server running in a Docker container. You use the application manager (appmgr) to manage PXE server docker hosting and functioning through Cisco IOS XR CLIs. This functionality lets you 'freeze' your booting environment in a Docker container instead of having to reinstall the environment for every new machine you want to boot, saving you the trouble of remembering the exact commands and sequences for a PXE boot.

Preboot Execution Environment (PXE) is a client-server interface that enables devices in a network to download the files (like boot image, configurations and so on) from PXE server.

The Client uses DHCP protocol to receive the PXE server details and uses TFTP or HTTP protocol to download the file.

Figure 3: Client-Server Connection



The PXE server docker feature enables the support of PXE/iPXE server functionality on the routers that run Cisco IOS-XR software.

This feature helps compute clusters (clients) to be upgraded from the Cisco 8201 top-of-rack router (server) that hosts the new image for software upgrade, thereby optimizing the operations and management bandwidth. PXE server docker feature is supported on both IPv4 and IPv6 addresses.

In this feature, the PXE server is installed on the Cisco 8201 router (server) in the form of a docker container that is managed by Cisco IOS-XR Application Manager (appmgr). The clients (routers or end-hosts such as Linux devices, VMs and so on) that are connected to this server can request and download the boot image.

The following services are packaged in a single PXE server docker container:

- DHCP
- HTTP (iPXE)
- TFTP (PXE)

These services are used for initial exchange of information and transferring the image between the client and the server.

The PXE server docker feature is available as part of the optional RPM—**xr-pxeserver**. This optional RPM contains:

- Executables for **pxe_svr_mgr** Cisco IOS-XR process.
- PXE server docker image —**pxe-server-docker.rpm**.

When the optional **xr-pxeserver** RPM is installed, the unsigned docker image (**pxe-server-docker.rpm**) is placed in the appmgr images directory **/pkg/opt/cisco/XR/appmgr/images/pxe-server-docker.rpm**, by the system.

- Helper scripts — **install_pxeserver.py** and **uninstall_pxeserver.py** placed under “**/pkg/bin/**” is used for installing and uninstalling **pxe-server-docker.rpm** from the application manager.



Note The helper scripts are used to manually install or uninstall **pxe-server-docker.rpm** and perform the installation or cleanup instead of using the application manager commands.

Behavioral Specifications

- The Cisco IOS XR DHCP proxy, server, and relay features for both IPv4 and IPv6 are not supported when the PXE server docker container is installed and running on the router.
- PXE server docker is supported only for BVI interfaces on its secondary IPv4 address. The PXE server docker is not supported on the primary IPv4 address of a BVI interface.
- Only one instance of PXE server docker container is supported to run on the router at a given time. Running multiple instances of PXE server docker container on multiple BVI interfaces in parallel is not supported and results in undefined behavior.
- Third-party application RPMs with the application name as “**pxe-server**” must not be installed along with this feature.
- Synchronizing between RPs for large files (iso images) take significant time (approximately 10 mins). If the system performs RPFO immediately after copying a large file to the application folder (**/harddisk:/mirror/server/images**), then these files should be copied again to the current active RP manually.
- The application state is not maintained after an upgrade. If the application is moved to STOP state and then updated using new version of Cisco IOS-XR RPM, then the new version of the application starts again automatically.
- Uninstalling the optional RPM (**xr-pxeserver**) does not stop or remove the PXE server docker container. User has to manually stop the PXE server docker container and uninstall the **pxe-server-docker.rpm** using the application manager commands.

Hosting and Activating the PXE Server Docker on Cisco 8201 Router using Application Manager

To host and activate the PXE server docker container application on Cisco 8201 router using application manager, follow these steps:

Procedure

Step 1 Install the optional RPM **xr-pxeserver** on the router:

```
Router#install package add xr-pxeserver
Router#install apply restart
Router#install commit
```

When the optional RPM- **xr-pxeserver** is installed and activated, the **pxe_svr_mgr** process is instantiated. The **pxe_svr_mgr** process handles installation and updating the PXE server docker RPM (**pxe-server-docker.rpm**) with application manager. Also, when the optional RPM **xr-pxeserver** is upgraded, the **pxe_svr_mgr** process checks for the new version of the **pxe-server-docker.rpm**. If there is a change in the version number, then it updates the existing version on the router to the new docker RPM version and re-launches the PXE server docker container for the changes to reflect.

Note

After activating the **xr-pxeserver** RPM package, the **pxe_svr_mgr** process installs the **pxe-server-docker.rpm** with application manager only when the ongoing install operation is committed and no more install operations are pending.

Step 2 Verify the PXE server docker container application package installed— Use the following command to verify the package installed:

```
Router#show appmgr packages installed
Package
-----
pxe-server-2.1.0-ThinXR.x86_64
```

Step 3 Create the following folder structure in the **/misc/disk1/** path and copy the **dhcp.conf** and **image.iso** files to their respective folders, as shown below:

```
/server
|---- config
|    |---- dhcpd.conf
|    |---- dhcpd6.conf
|---- images
|    |---- Image-boot.iso
----logs
```

Note

The PXE server docker container uses the **"/server/"** folder for its operations. This path is mounted to PXE server docker using the application manager configuration.

In case of a dual RP system, it is recommended to create this directory structure under **"/misc/disk1/mirror/"**. This automatically syncs the PXE server related files to the standby RP node. Therefore, all these files will be available on the new active RP node after RPFO. Otherwise, the user must create the directory structure again and copy all the necessary files for the PXE server docker container.

- Step 4** Configure and activate the PXE server docker container application— Use the following set of commands to configure and activate the PXE server docker container application on the interface BVI301:

```
Router#config
Router(config)#appmgr
Router(config-appmgr)#application pxeserver
Router(config-application)# activate type docker source pxe-server docker-run-opts "-it --restart
always --cap-add=NET_ADMIN --net=host --log-opt max-size=20m --log-opt max-file=3 -v
/misc/disk1/mirror/server:/server " docker-run-cmd "-i BV301 -4 172.16.0.0/12 -6 2001:DB8::/48 -l
/server/images -t"
Router(config-application)#commit
!
!
where,
--cap-add=NET_ADMIN --net=host (mandatory)
-i <interface> (mandatory)
-4 <secondary ipv4 address of BVI>
-6 <ipv6 address of BVI>
-l <location of image stored> (default /server/images)
-t <l - tftp enabled, 0 - tftp disabled>
```

- Step 5** Verify the PXE server docker container status—Use the following command to verify the PXE server docker container status:

```
Router(config)#appmgr application exec name pxeserver docker-exec-cmd status
```

dhcpd	RUNNING	pid 94, uptime 0:00:05
dhcpd6	RUNNING	pid 95, uptime 0:00:05
monitor	RUNNING	pid 96, uptime 0:00:05
nginx	RUNNING	pid 97, uptime 0:00:05
syslogd	RUNNING	pid 98, uptime 0:00:05
tftp-hpa4	RUNNING	pid 101, uptime 0:00:05
tftp-hpa6	RUNNING	pid 104, uptime 0:00:05

C

What to do next

The PXE server docker container is active and now the clients can download the boot image and the configuration file from the PXE server (Cisco 8201 router).

CPU-Based Packet Generator

Table 8: Feature History Table

Feature Name	Release Information	Feature Description
CPU-Based Packet Generator	Release 25.1.1	Introduced in this release on: Fixed Systems (8010 [ASIC: A100])(select variants only*) This feature is supported on Cisco 8011-4G24Y4H-I routers.

Feature Name	Release Information	Feature Description
CPU-Based Packet Generator	Release 24.4.1	Introduced in this release on: Fixed Systems (8200 [ASIC: P100], 8700 [ASIC: P100, K100])(select variants only); Modular Systems (8800 [LC ASIC: P100])(select variants only*) *This feature is supported on: • 8212-48FH-M • 8711-32FH-M • 8712-MOD-M • 88-LC1-36EH • 88-LC1-12TH24FH-E • 88-LC1-52Y8H-EM
CPU-Based Packet Generator	Release 24.2.1	You can now use a CPU-based packet generator for IOS-XR routers to simplify the diagnostic process for routers experiencing problems. This tool allows you to generate a wide range of traffic streams directly within the production environment without physically isolating the routers and moving them to a lab setup. This tool is beneficial in environments that use routers from different vendors or different models from the same vendor. The feature introduces the packetgen command with different options to generate different types of packets.

Need for CPU-Based Packet Generator

Diagnosing network problems in production environments, such as traffic drops and mis-forwarding issues, is crucial for network management. Traditionally, routers are physically isolated for debugging, requiring moving equipment into lab environments with traffic generators. The CPU-Based Packet Generator can be used in the production environment, eliminating the need to isolate the routers to a lab environment for troubleshooting purposes.

Benefits of CPU-Based Packet Generator

- **Versatile Traffic Crafting:** Create complex nested packets, such as IPinIPinIPinIP, to test and diagnose a variety of scenarios.
- **In-Production Diagnosis:** Directly diagnose routers in a problem state without disrupting the network setup.

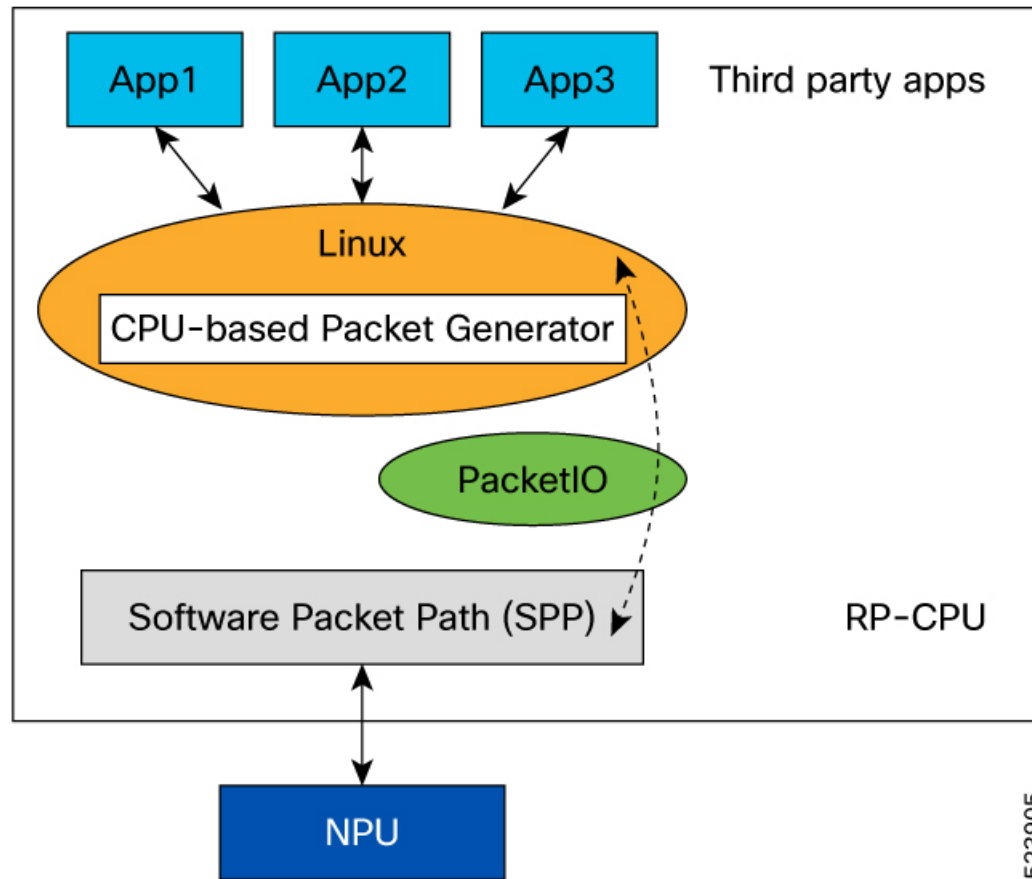
Restrictions of CPU-Based Packet Generator

- CPU-based packet generators are not optimized for high-speed packet processing; therefore, they may not match the performance of NPU-based packet generators.
- CPU-based packet generators can potentially introduce higher CPU loads during operation, which may affect the router performance.
- The probe packet rate is 40 kpps for Cisco 8000 Series Routers.

Topology of CPU-Based Packet Generator

The following diagram depicts the software architecture of CPU-based packet generator.

Figure 4: Architecture of CPU-Based Packet Generator



523905

The Cisco IOS-XR PacketIO serves as a host for third-party applications on the XR platform, with PacketIO infrastructure facilitating packet transport and interactions between Linux and XR environments. Leveraging this existing infrastructure, the CPU-based packet generator is implemented as a Linux application and packaged within the supported XR platform base image, ensuring seamless distribution.

The Linux infrastructure maintains a database of all XR interfaces including bundles. The CPU-based packet generator is used to send a specific packet type over a chosen interface.

Capabilities of CPU-based Packet Generator

- **Support different packet types:** The CPU-based packet generator supports various packet types, including:
 - ARP
 - TCP
 - UDP
 - GRE
 - MPLS

- IPinIP
- ICMPv4
- ICMPv6
- **Corrupt or error packet generation:** There are times when routers receive packets that are either corrupted or contain errors for various reasons. To identify and troubleshoot these issues, it becomes necessary to generate similar packets that can be used for debugging purposes. The CPU-based packet generator can create these packets and aid debugging.

Examples include:

- IPv4 packet with TTL 0
- IPv4 packet with wrong checksum
- IPv4 packet with mismatch between IP option length field and the IP header

How to Use CPU-based Packet Generator?

You can use CPU-based packet generator using:

- **CLI:** Use the **packetgen** command with different options to run the tool from XR bash environment. As the XR interfaces show up as Linux interfaces in bash environment, you can directly use the XR interface names.
- **pcap file:** Use an already captured pcap file in production routers and replay it.

packetgen -i interface_name -pcap pcap_file

CLI Options

The following table outlines the different options available for the **packetgen** command.

Table 9: Packetgen CLI Options

Option	Description
-accounting	Turn on accounting for packets. Only works if packets come back to the packet generator.
-arp-destination-hw-address string	ARP target hardware address (default: uses interface MAC address)
-arp-destination-ip-address string	ARP target IP address (default: 127.0.0.1 or ::1)
-arp-operation uint	ARP operation (1: request, 2: reply , 3: rarp)
-arp-source-hw-address string	ARP sender hardware address (default : uses interface MAC)

Option	Description
-arp-source-ip-address string	ARP sender IP address (default: uses interface IP)
-burst int	Number of packets to be injected at a time. To be used in conjunction with -sleep.
-count int	Number of packets to be generated.
-data-type string	constant, incrementing, random (default: no payload)
-ethernet-dmac string	Destination MAC address (default: ff:ff:ff:ff:ff:ff)
-ethernet-smac string	Source MAC address (default: use interface MAC address)
-file string	Write packets to file
-gre	Enable GRE
-gre-checksum-present	Enable GRE checksum present bit
-gre-key-present	Enable GRE key present bit
-gre-over-mpls	Enable GRE over MPLS
-gre-protocol uint	Set the protocol type of the GRE payload (default: 0x0800 (IP))
-gre-seq-present	Enable GRE sequence number present bit
-gre-version uint	Set the GRE version number (default 0)
-header string	Custom header for all packets
-hex	Print hex dump of packets
-i string	Interface name for packet injection
-icmp-code uint	ICMP code (default: 0)
-icmp-type uint	ICMP type (default: 0)
-inc-dmac	Increment destination MAC
-inc-smac	Increment source mac
-inner-ethernet-dmac string	Inner Ethernet destination MAC address (default: ff:ff:ff:ff:ff:ff)
-inner-ethernet-smac string	Inner Ethernet source MAC address (default: ff:ff:ff:ff:ff:ff)
-inner-ip-checksum uint	Inner IP checksum (default: compute checksum automatically)
-inner-ip-dont-fragment uint	Set inner IP Don't Fragment flag as 1

Option	Description
-inner-ip-dst string	Inner destination IP address (default: 127.0.0.1 or ::1)
-inner-ip-flow-label uint	Inner IPv6 Flow Label value (default: 0)
-inner-ip-frag-offset uint	Inner IP fragment offset in units of 64-bits (e.g. 1 = 64 bits)
-inner-ip-protocol string	Inner IP protocol . Supports protocol text (TCP, UDP) and code (63 for TCP) (default: TCP)
-inner-ip-src string	Inner source IP address (default: 127.0.0.1 or ::1)
-inner-ip-tos uint	Inner IP Type Of Service (TOS) value (default: 0)
-inner-ip-traffic-class uint	ip-traffic-class (traffic-class) value (default: 0)
-inner-ip-ttl uint	Inner IP time to live (ttl). (Default ttl = 64
-inner-ip-version int	Inner IP version (default: 4)
-inner-vlan-id uint	Inner VLAN id (default: 0)
-inner-vlan-tpid uint	Inner VLAN ethernet type (default: 33024 :Dot1Q)
-inner-vlan-vpri uint	Inner VLANpriority (default: 0
-ip-checksum string	IP checksum (default: compute checksum automatically)
-ip-dont-fragment string	Set IP flag -ip-dont-fragment 0 -> 000 Nothing set -ip-dont-fragment 1 -> 001 More Fragments -ip-dont-fragment 2 -> 010 Dont Fragment -ip-dont-fragment 4 -> 100 set reserved bit
-ip-dst string	Destination IP address (default: 127.0.0.1 or ::1)
-ip-flow-label string	IPv6 Flow Label value (default: 0)
-ip-frag-offset string	Fragment offset in units of 64-bits (1 = 64 bits)
-ip-protocol string	IP protocol. Supports protocol text (TCP, UDP, GRE, VXLAN, ICMP, NDP) and code (63 for TCP) (default: TCP)
-ip-src string	Source IP address (default: use interface ip)
-ip-tos string	IP Type Of Service value (default: 0)
-ip-traffic-class string	IP traffic class (traffic-class) value (default: 0)
-ip-ttl string	IP time to live (ttl). (Default ttl = 64
-ip-version string	IP version should always be set for accurate IP packet creation, ip version (default: 4).

Option	Description
-mpls-exp string	Comma separated MPLS EXP (Experimental) value (default: 0)
-mpls-label string	Comma separated list of Multiprotocol Label Switching (MPLS) labels to be added to the packet. Specified from top to bottom
-mpls-ttl string	Comma separated MPLS TTL (Time To Live) value (default: 64)
-ndp string	Specify the neighbor discovery protocol: nbr-solicit, nbr-advt
-ndp-target-address string	NDP target address (default: for advertisement source IP, for solicitation destination IP)
-pcap string	File to replay pcap
-progress	Display a progress bar
-seed int	Seed for pseudo random payload generator
-size int	Size of payload
-sleep string	Time duration to sleep during each burst. To be used together with -burst.
-stdout	Print packets to stdout
-tcp-dport int	TCP destination port (default: 40000)
-tcp-flags string	Set TCP control flags: • U (Urgent): Indicates that the data should be processed urgently. • A (Acknowledgement): Acknowledges the receipt of data. • P (Push): Instructs the sender to push the data to the receiving application immediately. • R (Reset): Resets the connection. • S (Synchronize): Synchronizes sequence numbers to initiate a connection. • F (Finish): Indicates the sender has finished sending data and wants to terminate the connection.
-tcp-sport int	TCP source port (default: 40000)
-udp-dport int	UDP destination port (default: 40000)
-udp-sport int	UDP source port (default: 40000)
-vlan-id uint	VLAN id (default: 0)
-vlan-tpid uint	VLAN ethernet type (default: 33024 :Dot1Q)
-vlan-vpri uint	VLAN priority (default: 0
-vxlan-udp-dport int	UDP destination port for VXLAN (default: 4789)

Option	Description
-vxlan-udp-sport int	UDP source port for VXLAN (default: 0)
-vxlan-vni uint	VXLAN VNI (default: 0)

Sample Commands

This section lists sample commands for some common packet types.

Table 10: Sample Packetgen Commands

Packet Type	Sample Command
ARP	packetgen -i enp0s8 -ip-ttl 32 -arp-operation 1 -progress -count 10000 -inc-smac -arp-destination-ip-address 192.168.56.1
TCP	packetgen -i enp0s8 -ip-ttl 32 -tcp-sport 40000 -progress -count 10000 -inc-smac
UDP	packetgen -i enp0s8 -ip-ttl 32 -udp-sport 40000 -progress -count 10000 -inc-smac
ICMP - PING	packetgen -i enp0s8 -ip-ttl 32 -icmp-type 8 -progress -count 10000 -ip-dst 192.168.56.1
GRE	packetgen -i enp0s8 -ip-ttl 32 -gre -count 100 -inner-ip-ttl 32 -tcp-sport 3222 -progress
IP in IP	packetgen -i enp0s8 -count 100 -tcp-sport 3222 -progress -ip-src="1.1.1.1,2.2.2.2"
ETHER-IP	packetgen -i enp0s8 -ip-ttl 32 -count 100 -inner-ip-version 6 -tcp-sport 3222 -progress -inner-ethernet-smac ff:ff:ff:ff:ff:ff
VLAN	packetgen -i enp0s8 -ip-ttl 32 -tcp-sport 40000 -progress -count 10000 -inc-smac -vlan-id 2
QinQ	packetgen -i enp0s8 -ip-ttl 32 -tcp-sport 40000 -progress -count 10000 -inc-smac -vlan-id 2 -inner-vlan-id 2
VXLAN	packetgen -i enp0s8 -ip-ttl 32 -tcp-sport 40000 -progress -count 10000 -inc-smac -vxlan-vni 3 -vxlan-udp-sport 4444 -inner-ip-version 4 -inner-ethernet-smac ff:ff:ff:ff:ff:ff -data-type constant
NDP	packetgen -i enp0s8 -ip-version 6 -ndp nbr-advt -count 100 -ip-checksum 1 -progress
MPLS	packetgen -i enp0s8 -ip-version 4 -mpls-label 1,2,3,4,5 -tcp-sport 4556 -count 1000 -progress

Command Example

This section shows an example command to send an ICMP ping request from source address 10.0.0.1 to destination address 10.0.0.2 via interface Hu0_0_0_25.

```
Router# bash
[ios:~]$ packetgen -i Hu0_0_0_25 -ip-ttl 32 -progress -count 50 -icmp-type 8 -ip-dst 10.0.0.2
-ip-src 10.0.0.1 --ethernet-smac 78:c5:51:84:48:c4 --ethernet-dmac 00:00:00:1e:ca:fc
INFO[0000] [ETH IP ICMP]
INFO[0000] Setting SRC IP to 10.0.0.1
```

```

INFO[0000] Setting DST IP to 10.0.0.2
INFO[0000] Opening Handle Hu0_0_0_25
INFO[0000] Opened Handle Hu0_0_0_25
INFO[0000] Starting Packet Injection
Sending Packets... 2% | | (1/50, 254 packet/s) [0s:0s] /* Truncated output. */

```

```

Address Age Hardware Addr State Type Interface
10.0.0.1 - 78c5.5184.48c4
Interface ARPA HundredGigE0/0/0/25
10.0.0.2 00:50:23 0000.001e.cafc Dynamic ARPA HundredGigE0/0/0/25

```

Source stats:

Stat Name	Port Name	Control Packet Tx.	Control Packet Rx.	Ping Reply Tx.
20.0.0.2/				
Card01/Port01	Ethernet - VM - 001	51	51	50

Interface stats:

Input	Punt	XIPC	InputQ	XIPC	PuntQ
ClientID	Drop/Total	Drop/Total	Cur/High/Max	Cur/High/Max	Cur/High/Max
ipv6_icmp	0/0	0/0	0/0/1000	0/0/1000	0/0/1000
icmp	0/50	0/0	0/15/1000	0/0/1000	0/0/1000